

SystemVerilog: язык, RTL и проверка проектов

Самостоятельная учебная методичка

IEEE 1800-2023

Редакция 2026-09-23

От основ аппаратного описания до RTL, тестбенчей и библиотечного моделирования. Самостоятельный учебный текст, примеры и упражнения. Для точной нормативной грамматики обращайтесь к IEEE 1800.

Оглавление

01. Аппаратное мышление и устройство языка	4
02. Лексика, литералы и четыре состояния	6
03. Модули, порты и иерархия	8
04. Время и планировщик симуляции	10
05. Сети, переменные и встроенные типы	12
06. Операции, размер и знаковость	14
07. typedef, enum, структуры и объединения	16
08. Массивы, очереди и методы	18
09. Совместимость и приведение типов	20
10. Присваивания и процессы RTL	22
11. Условия, выбор и циклы	24
12. Функции, задачи и время жизни	26
13. Пакеты, области видимости и bind	28
14. Параметры и generate	30
15. Интерфейсы и modport	32
16. Препроцессор и единицы компиляции	34
17. Комбинационная логика и арифметика	36
18. Регистры, сброс и конвейеры	38
19. Память, ROM и очереди FIFO	40
20. Конечные автоматы и протоколы	42
21. Тактирование, CDC и переносимость	44
22. Сквозной проект: синхронный FIFO	46
23. Тестбенчи, clocking и ввод-вывод	51
24. Параллельные процессы и синхронизация	53

25. Классы и объектная модель	55
26. Случайные воздействия и ограничения	57
27. Assertions, sequences и properties	59
28. Функциональное покрытие	61
29. DPI, VPI и границы UVM	63
30. Примитивы, силы и пользовательские сети	65
31. specify, задержки и библиотечные модели	67
32. Библиотеки и конфигурации	69
33. Стандарты, изменения 2023 и рабочий процесс	71
A. Матрица синтезируемости	73
B. Справочник ключевых слов и системных средств	75
C. Упражнения и ответы	78
D. Соответствие исходному PLAN.md	81
E. Источники и порядок проверки фактов	95
F. Словарь и предметный указатель	96

01. Аппаратное мышление и устройство языка

SystemVerilog описывает одновременно структуру цифровой системы, её поведение во времени и окружение, которое проверяет это поведение. Один файл может содержать формулу сумматора, регистр с тактовым входом и класс генератора транзакций. Синтаксическая корректность всех трёх описаний не означает, что все они превращаются в транзисторы одним и тем же инструментом.

От программы к схеме

В обычной программе выражение вычисляется, когда до него дошёл поток исполнения. В аппаратном описании `assign y = a & b;` задаёт постоянно действующую связь: при изменении входов пересчитывается выход. Два таких присваивания существуют параллельно. Порядок их записи в файле не создаёт последовательность тактов.

Регистр хранит состояние между событиями. Комбинационная схема вычисляет результат по текущим входам. Если представить устройство как граф, регистры будут границами между участками комбинационной логики. Задержка реального участка определяется технологией и разводкой; запись + сама по себе не задаёт наносекунды.

Полный небольшой модуль синтезируемого RTL:

```
module enabled_register (
  input  logic      clk, rst_n, en,
  input  logic [7:0] d,
  output logic [7:0] q
);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= '0;
    else if (en) q <= d;
endmodule
```

Это восемь триггеров с асинхронным сбросом и разрешением записи. Отсутствие `else` после проверки `en` означает сохранение состояния. Условия синтеза `enable` могут дать специальный вход триггера или мультиплексор перед ним. Позже мы разберём, почему здесь используется `<=`, а не `=`.

Три этапа обработки

1. **Компиляция:** препроцессор раскрывает директивы, компилятор разбирает синтаксис, имена и типы. Пакет должен быть доступен до использования его объявлений.
2. **Элаборация:** выбирается верхний модуль, создаются экземпляры, подставляются параметры, раскрывается `generate`, проверяются соединения.
3. **Моделирование:** процессы работают в дискретном времени по правилам планировщика.

Синтез использует элаборированное описание допустимого подмножества языка и строит логическую сеть. После оптимизации и технологического отображения можно получить `netlist`. Размещение, трассировка и статический временной анализ — следующие этапы проектирования, а не операторы SystemVerilog.

Уровни описания и элементы проекта

Элемент	Что выражает	Обычное применение
<code>module</code>	Иерархический аппаратный блок	RTL, netlist, тестбенч

Элемент	Что выражает	Обычное применение
interface	Группа сигналов и протокол доступа	Соединение блоков
package	Общие типы, константы, подпрограммы	Организация исходников
primitive	Пользовательский примитив с таблицей	Модели ячеек
program	Проверяющий процесс с Reactive-семантикой	Специализированные тестбенчи
checker	Повторно используемый набор проверок	Assertions
config	Выбор библиотечных реализаций	Элаборация проекта

Поведенческая модель описывает «что получается», структурная — «из чего собрано». RTL описывает передачу и преобразование данных между регистрами. Это не три разных языка: один модуль может соединять экземпляры, содержать формулы и тактовые процессы.

История и маршрут обучения

SystemVerilog развивает Verilog, добавляя типы, специализированные процессы RTL, интерфейсы и средства проверки. Редакция IEEE 1800-2009 объединила описание базового Verilog и расширений SystemVerilog; последующие редакции уточняли и расширяли язык. В этой методичке основа — [IEEE 1800-2023](#). Сведения о переходе с 2017 на 2023 собраны в [главе 33](#).

Сначала изучайте обычные модули и значения, затем временную семантику и RTL. Классы и constrained random полезны после того, как вы способны объяснить, на каком фронте устройство принимает вход и когда его выход становится доступен. Наличие большого тестбенча не исправляет неопределённый контракт устройства.

Упражнение 01

У регистра выше $en=0$, d изменяется десять раз между фронтами clk . Сколько раз должен измениться q , если сброс не активен? Как изменится ответ для `assign q = d;`? Ответы — в [приложении С](#).

02. Лексика, литералы и четыре состояния

Лексика определяет, как текст превращается в токены. SystemVerilog различает регистр: Data и data — разные имена. Пробелы, табуляция и переводы строк обычно разделяют токены; исключения важны для макросов и экранированных идентификаторов. Комментарии // продолжаются до конца строки, /* ... */ охватывают несколько строк и не вкладываются друг в друга.

Идентификаторы и слова

Обычный идентификатор начинается с буквы или _, далее допускает цифры и \$. Имена, начинающиеся с \$, используются системными задачами и функциями, например \$display. Ключевое слово нельзя использовать как обычное имя. Экранированный идентификатор начинается с обратной косой черты и заканчивается **пробельным символом**, который не входит в имя:

```
wire \data[3] ; // Одно имя с квадратными скобками, не выбор бита.
```

Такие имена часто встречаются в автоматически созданных netlist. Для учебного RTL удобнее обычные идентификаторы. Директивы препроцессора начинаются с обратного апострофа: `include, `define; это не одиночная кавычка числа.

Значения 0, 1, X и Z

0 и 1 — двоичные значения, x — неизвестность, z — высокоимпедансное состояние драйвера. Это значения модели, а не четыре физических напряжения. Неинициализированная четырёхсостоянийная переменная обычно содержит X, сеть без драйвера — Z. Для двухсостоянийного интегрального типа начальное значение — 0.

X помогает обнаруживать неопределённое состояние, но симуляция не является точной моделью метастабильности. Z полезно на общей шине; внутри FPGA трёхстабильная сеть обычно заменяется иной логикой или не поддерживается в таком виде.

Выражение	Результат	Причина
1'b0 & 1'bx	0	Нулевой вход определяет AND
1'b1 & 1'bx	x	Ответ зависит от неизвестного входа
1'b1 1'bz	1	Единица определяет OR
1'bx == 1'bx	x	Обычное равенство допускает неизвестный ответ
1'bx === 1'bx	1	Case equality сравнивает и X/Z

Целые литералы

Полная форма: размер [s] основаниецифры. Размер измеряется в битах, основания — b, o, d, h; s задаёт знаковость. Подчёркивания облегчают чтение:

```
logic [7:0] mask = 8'b1010_0011;
logic signed [7:0] delta = -8'sd3;
logic [11:0] address = 12'hA50;
logic [31:0] all_ones = '1;
```

8'hFF — беззнаковое 255, 8'shFF — знаковое -1. Минус — отдельная унарная операция. Безразмерное десятичное 42 имеет знаковый интегральный тип и размер не менее 32 бит; нельзя считать любой

безразмерный литерал ровно 32-битным. Литералы '0, '1, 'x, 'z заполняют требуемый контекст соответствующим битом. В самодостаточном контексте такой литерал имеет один бит: это существенно внутри конкатенаций. Слишком широкое число усекается до явно указанного размера.

Вещественные числа, строки и время

1.25, 3.0e-6 — вещественные литералы; они подходят моделированию, но не задают автоматически аппаратный floating-point блок. 10ns — литерал времени. Двойные кавычки обозначают строку, \n, \t, \", \\ — escape-последовательности. Строковый литерал можно использовать как текст или как упакованное представление байтов в соответствующем контексте. Тип string имеет динамическую длину.

В редакции 2023 появились многострочные литералы с тройными двойными кавычками. Для переносимого RTL они обычно не нужны; язык парсера должен поддерживать 2023. Детали новых средств и первичные ссылки находятся в [главе 33](#).

Атрибуты

(* имя = значение *) прикрепляет метаданные к объявлению или другой разрешённой конструкции. Например, имя атрибута может просить синтезатор сохранить сигнал. Сам синтаксис стандартизован, но смысл многих имён задаёт производитель инструмента. Нельзя считать произвольный атрибут переносимой семантикой аппаратуры.

Упражнение 02

Чему равны 8'hFF, 8'shFF, 4'b1x00 === 4'b1x00? Почему присваивание logic [15:0] x = '1; отличается от logic [15:0] y = 1;?

03. Модули, порты и иерархия

Модуль — шаблон структуры. Экземпляр модуля — конкретный узел иерархии со своими сигналами и состоянием. Два экземпляра одного счётчика работают одновременно и не разделяют регистры, если это явно не задано соединениями.

Определение и экземпляр

Полные единицы компиляции с параметром ширины:

```
module add_words #(parameter int unsigned W = 8) (  
    input  wire [W-1:0] a, b,  
    output wire [W:0] sum  
);  
    assign sum = {1'b0, a} + {1'b0, b};  
endmodule  
  
module arithmetic_top (  
    input  wire [7:0] lhs, rhs,  
    output wire [8:0] result  
);  
    add_words #(.W(8)) u_add (.a(lhs), .b(rhs), .sum(result));  
endmodule
```

У `u_add` ширина определяется во время элаборации. `a` и `b` — имена формальных портов, `lhs` и `rhs` — подключаемые выражения верхнего уровня. Соединения не вызывают функцию: сумматор присутствует в схеме всё время.

ANSI-стиль объявляет направление и тип прямо в заголовке. Старый non-ANSI-стиль перечисляет имена в заголовке и объявляет их в теле. Он нужен при чтении наследуемого кода, но смешивание стилей усложняет проверку интерфейса.

Направление, тип данных и вид объекта

У порта различаются направление (`input`, `output`, `inout`, `ref`), вид объекта (сеть или переменная) и тип данных. Для обычных соединений поток определяется направлением. `ref` создаёт ссылку на переменную и имеет строгие правила типов; это не обычная аппаратная двунаправленная шина.

У `input logic` вид порта по умолчанию сетевой, если явно не указано `var`. У `output logic` с явно заданным типом данных по умолчанию получается переменная. У `inout` нужен сетевой объект. Чтобы убрать сомнения, можно написать `input wire logic [7:0] a` или `output var logic [7:0] q`. Само слово `logic` не означает «регистр» и не всегда означает переменную.

Преобразования на границе порта могут менять ширину и знаковость. Предупреждение о несовпадении размеров следует разбирать, а не глушить: потерянный старший бит часто является потерянным переносом. У распакованных массивов и интерфейсов правила соединения строже, чем у обычных интегральных векторов.

Способы соединения

Запись	Смысл	Практическое замечание
<code>.a(lhs)</code>	Явное имя порта и выражение	Удобно для проверки соединений
<code>.a</code>	Подключить одноимённый объект <code>a</code>	Тип и область поиска должны подходить

Запись	Смысл	Практическое замечание
<code>.*</code>	Подключить оставшиеся подходящие одноимённые порты	Изменение имён влияет на соединения
<code>.a()</code>	Оставить порт неподключённым	Намерение видно при ревью
<code>(lhs, rhs, result)</code>	Позиционные соединения	Зависит от порядка портов

`.*` не создаёт недостающие сигналы и не освобождает от проверки типов. В крупных блоках явные именованные соединения лучше показывают намерение. Массив экземпляров задаёт несколько однотипных устройств; `generate` позволяет различать их параметры и условно включать структуру.

Имена, вложенность и особые объявления

Иерархическое имя вроде `top.u_add.sum` указывает объект конкретного экземпляра. Такие обращения удобны в тестбенче, но жёстко привязывают код к иерархии. Общие типы лучше выносить в пакет, доступ к сигналам — в порты и интерфейсы.

Язык допускает вложенные определения модулей и `extern`-прототипы. Это средства организации и раздельного описания интерфейса; практическую поддержку проверяют по компилятору. Прототип объявляет интерфейс, но не создаёт реализацию блока. Директива `default_nettype none` помогает ловить опечатки, которые иначе могут породить неявную однобитную сеть.

`alias` объединяет сетевые объекты или их части в одну электрическую связь; это двунаправленная идентичность сети, а не переименование переменной и не две операции `assign`. Например, фрагмент `wire a, b; alias a = b;` требует поддержки сетевых псевдонимов инструментом. Детали разрешения драйверов — в [главе 30](#).

Упражнение 03

Почему у суммы двух 8-битных беззнаковых чисел нужен 9-битный выход? Что изменится, если поменять порядок объявления портов при именованном и позиционном соединении?

04. Время и планировщик симуляции

Моделирование — обработка очередей событий. Симулятор перескакивает к следующему моменту, в котором есть работа, и может многократно пересчитывать процессы внутри одного значения времени. Такое продвижение без изменения времени часто называют дельта-циклами. Оно позволяет сигналу пройти цепочку комбинационных зависимостей.

Единица времени и точность

В модуле или интерфейсе объявления времени помещают до остальных элементов:

```
module time_demo;
    timeunit 1ns;
    timeprecision 1ps;
    logic clk = 0;
    always #5ns clk = ~clk;
    initial begin
        #12.345ns;
        $display("time=%0t", $realtime);
        $finish;
    end
endmodule
```

`timeunit` задаёт масштаб безразмерных задержек, `timeprecision` — точность квантования задержек данного контекста. Глобальная точность симуляции учитывает самую мелкую требуемую точность проекта. `$time` возвращает целое время, `$realtime` — вещественное; формат `%t` дополнительно зависит от `$timeformat`. Директива ``timescale` поддерживается для старых исходников, но её действие зависит от порядка текста. Локальные объявления делают модель понятнее.

Области одного временного слота

Ниже — рабочая карта, а не обещание одного линейного прохода. События могут возвращать обработку к более ранним областям того же временного слота.

Область	Характерная работа
Preponed	Выборка значений для конкурентных assertions
Active	Выполнение обычных модульных процессов, blocking assignments
Inactive	Возобновление модульных процессов после #0
NBA	Обновления от неблокирующих присваиваний
Observed	Оценка конкурентных свойств по выбранным значениям
Reactive	Процессы program, действия конкурентных assertions
Re-Inactive, Re-NBA	Аналогичные очереди для реактивного набора
Postponed	<code>\$strobe</code> , <code>\$monitor</code> , наблюдение установившихся значений

Между этими областями стандарт задаёт и точки взаимодействия PLI: Pre-Active, Pre-NBA, Post-NBA, Pre-Observed, Post-Observed, Pre-Re-NBA, Post-Re-NBA, Pre-Postponed. Для повседневного RTL важнее понимать выборку, Active и NBA, чем запоминать API-области.

Неблокирующее присваивание

Внутри тактового процесса правая часть $q \leftarrow d$; вычисляется при выполнении оператора; обновление q планируется в NBA. Поэтому два регистра могут одновременно принять старые значения друг друга:

```
// Фрагмент внутри модуля; a и b объявлены logic.  
always_ff @(posedge clk) begin  
    a <= b;  
    b <= a;  
end
```

Если перед фронтом $a=1$, $b=0$, после NBA получим $a=0$, $b=1$. С блокирующим = во втором присваивании уже читалось бы новое a . Порядок разных процессов в Active не является способом задать приоритет.

Гонки и наблюдение

Если DUT читает $data$ на `posedge clk`, а тестбенч блокирующим присваиванием меняет $data$ на том же фронте, результат может зависеть от порядка выполнения процессов. Переставлять строки между файлами — ненадёжное «исправление». Простое учебное решение — подавать стимулы на противоположном фронте, а результат наблюдать после обновления состояния. Для сложных протоколов служит `clocking`.

`#0` не означает «дождаться всех NBA»: `Inactive` предшествует NBA. `$display` в тактовом процессе может вывести старое значение, а `$strobe` — установившееся в конце временного слота. Проверки должны заранее определять, какое значение им нужно.

Задержки `#` описывают модель и тестбенч. В обычном RTL они не превращаются в заданный физический интервал. Для задержки на N тактов нужны N состояний или регистров и спецификация частоты.

Упражнение 04

До фронта $a=2$, $b=7$. Определите значения после $a \leftarrow b$; $b \leftarrow a$; и после $a = b$; $b = a$; . Объясните, почему `#0` перед проверкой не гарантирует чтение обновлённого неблокирующим присваиванием регистра.

05. Сети, переменные и встроенные типы

Тип данных описывает допустимые значения и операции. Вид объекта определяет, как значение хранится или получается от драйверов. Эти измерения независимы: `wire logic [7:0] bus;` — сеть с четырёхсостоянийным векторным типом данных, а `logic [7:0] state;` — переменная того же типа данных.

Сеть и переменная

Сеть (`wire`, `tri` и другие) отражает значения драйверов, используя функцию разрешения. У обычной `wire` два равносильных противоречащих драйвера дают X. Переменная хранит последнее присвоенное значение; она не выполняет электрическое разрешение нескольких независимых записей. Множественные записи могут быть запрещены правилами конструкции либо создавать гонку.

```
// Объявления внутри модуля.  
wire logic [7:0] resolved_bus;  
logic [7:0] register_value;  
bit ready_model;
```

`reg` — историческое имя четырёхсостоянийного типа переменной. Оно не гарантирует получение триггера: триггер выводится из поведения процесса. В новом коде обычно используют `logic`, сохраняя явный `wire` там, где существенна сетевая семантика. `var` явно указывает переменную. `uwire` помогает обнаруживать несколько драйверов.

Встроенные числовые типы

Тип	Бит	Состояния	Знаковость по умолчанию
<code>bit</code>	1	2	unsigned
<code>logic</code> , <code>reg</code>	1	4	unsigned
<code>byte</code>	8	2	signed
<code>shortint</code>	16	2	signed
<code>int</code>	32	2	signed
<code>longint</code>	64	2	signed
<code>integer</code>	32	4	signed
<code>time</code>	64	4	unsigned

`int unsigned` удобен для неотрицательных счётчиков тестбенча и параметров. Для регистра заданной аппаратной ширины предпочтительнее `logic [W-1:0]`. `real/realtime` соответствуют `double-precision` представлению, `shortreal` — `single-precision`. Они применяются в моделях; для синтеза вещественной арифметики требуется явно поддерживаемый инструментом путь.

Преобразование X/Z в двухсостоянийный тип заменяет их нулями. Поэтому `bit` на наблюдаемом выходе DUT способен скрыть неисправность сброса. Экономия памяти тестбенча не должна менять смысл проверки.

Векторы и выбор частей

```
logic [15:0] word;  
logic [0:15] forward_word;  
logic [3:0] nibble;  
// Фрагмент процесса:  
// nibble = word[7:4];  
// nibble = word[base +: 4];
```

В упакованном диапазоне левый индекс обозначает старшую позицию независимо от численного порядка индексов. Обычный part-select требует постоянных границ. Indexed part-select `base +: width` или `base -: width` допускает переменную базу и постоянную ширину. Выбор бита и части packed-вектора сам по себе беззнаковый, даже если исходный вектор был signed.

Выход за границы — ошибка проектирования. Для четырёхсостоянийного чтения неопределённого бита получится X; детали агрегатов зависят от типа элемента. Не следует строить функциональность на поведении некорректного индекса.

Область видимости, время жизни, инициализация

Область видимости отвечает на вопрос «где доступно имя», время жизни — «как долго существует объект». Переменные модуля живут всю симуляцию. Автоматические локальные переменные создаются заново для каждого вызова подпрограммы или входа в соответствующую область. Статические сохраняются между вызовами.

Инициализация статической переменной при объявлении происходит один раз, а автоматической — при каждом создании. `logic q = 0;` не эквивалентно сбросу, действующему при каждом включении системы. Поддержка начального состояния аппаратуры зависит от FPGA/ASIC и синтезатора.

Константы и прочие типы

`parameter` можно переопределить при создании экземпляра, `localparam` — нельзя. `const` запрещает последующие изменения переменной по правилам `const`, но не превращает её автоматически в элаборационную константу. `spesparam` используется в моделях задержек. `$` имеет специальный смысл неограниченной границы в разрешённых контекстах и не является универсальным числом «бесконечность».

`string` хранит последовательность байтов переменной длины; `chandle` — непрозрачный указатель для DPI; `event` — средство синхронизации. Агрегатные типы объединяют несколько значений; singular-типы противопоставляются распакованным структурам, объединениям и массивам. Интегральные типы допускают битовую интерпретацию, включая packed-структуры и packed-массивы.

Упражнение 05

Выберите объявления для 13-битного регистра, 32-битного программного счётчика тестбенча без X и общей линии с несколькими драйверами. Почему `const int n` не всегда подходит в диапазоне массива?

06. Операции, размер и знаковость

При чтении выражения нужно ответить на три вопроса: какой оператор выполняется, сколько бит участвует в вычислении и как эти биты интерпретируются по знаку. Математически верная формула может дать неправильную аппаратную арифметику из-за усечения промежуточного результата.

Семейства операций

Семейство	Операторы	Смысл
Арифметика	+ - * / % **	Числовые действия
Сравнение	< <= > >=	Однобитный результат
Равенство	== != === !== ==? !=?	Обычное, case и wildcard equality
Логика	! &&	Проверка истинности операндов
Побитовые	~ & ^ ~^ ~^	Отдельная операция над каждой позицией
Редукции	& ~& ~ ^ ~^	Один результат из всех битов вектора
Сдвиги	<< >> <<< >>>	Логический или арифметический сдвиг
Выбор	cond ? a : b	Условное выражение
Сборка	{a,b}, {N{a}}	Конкатенация и репликация
Множество	inside	Принадлежность списку значений/диапазонов

`a & b` и `a && b` различаются даже для обычных двоичных векторов. При `a=2'b10`, `b=2'b01` побитовое AND даёт `00`, а логическое AND — `1`. Унарная редукция `&a` отвечает, все ли биты равны единице.

`==` может вернуть `X`. `===` всегда даёт `0` или `1`, считая `X/Z` самостоятельными значениями. В `==?` `X/Z` правого операнда служат маской; левый операнд не получает такой симметричной роли. В RTL сравнение с `X` не означает детектор физической неизвестности. В тестбенче `!==` полезно, чтобы `X` не превратился в молчаливый пропуск.

Размер вычисления

Часть выражений имеет self-determined размер: например, элементы конкатенации определяют свои размеры до сборки. Другие получают размер из контекста присваивания и соседних операндов. Поэтому правило «плюс всегда имеет ширину только входов» недостаточно, но и надежда на последующее расширение уже усечённой части опасна.

```
// Фрагмент модуля, комбинаторная арифметика.  
logic [7:0] a, b;  
logic [8:0] sum;  
assign sum = {1'b0, a} + {1'b0, b};
```

Явное расширение делает перенос видимым независимо от читательских догадок о распространении контекста. Для произведения двух `W`-битных беззнаковых чисел нужно до `2W` бит результата. В конкатенации `{a + b}` сумма вычисляется в собственном контексте; скобки `{ }` не аналог обычных `( )`.

`$bits(T)` и `$bits(expr)` позволяют проверить размер. Сначала расширяйте операнды до требуемого типа, затем выполняйте действие. Приведение готового усечённого результата не восстанавливает потерянные биты.

Signed и unsigned

Для обычной бинарной интегральной арифметики смесь signed и unsigned часто приводит к беззнаковой интерпретации. `logic signed [7:0] s` может хранить -1, но сравнение с беззнаковым вектором требует особого внимания. Конкатенация, `bit-select` и `part-select` дают unsigned-результаты; `$signed` меняет трактовку битов, но не увеличивает их число.

```
logic signed [7:0] value;
logic signed [8:0] extended;
assign extended = {value[7], value};
```

В этой записи расширение знака выполнено явно, а signed-объявление приёмника задаёт дальнейшую интерпретацию. `>>>` заполняет старшие позиции знаком только для signed левого операнда. `>>` вдвигает нули. `<<<` не создаёт особого «знакового заполнения слева»; оба левых сдвига вдвигают нули справа.

Условие, множества и потоки

При однобитном неизвестном условии тернарный оператор для интегральных ветвей объединяет результаты побитно: одинаковые биты сохраняются, различающиеся становятся X. Процедурный `if` при неопределённом условии идёт в `else`. Это важный источник различия X-поведения двух внешне похожих записей.

`addr inside {[8'h20:8'h2f], 8'h80}` описывает набор. `case (...)` `inside` использует подобные правила выбора. `{>>{data}}` и `{<<8{data}}` — потоковые операции: они укладывают/переставляют группы битов. Вторая форма обращает порядок 8-битных групп, а не обязательно порядок битов внутри байта. Всегда фиксируйте размер группы и ожидаемый порядок на маленьком примере.

`let` объявляет параметризованное выражение в своей области видимости:

```
// Фрагмент внутри модуля.
let accepted(v, r) = v && r;
```

Это не хранилище и не процесс. Константное выражение вычисляется при элаборации; оно может вызывать допустимые константные функции, но не зависеть от текущего входа.

Приоритет и побочные эффекты

Практический порядок: унарные операции, степень, умножение/деление, сложение, сдвиги, отношения, равенство, побитовые, логические, условный выбор. Внутри семейств есть дополнительные уровни. Смешанные выражения лучше явно группировать: `(a == b) && enable`, `(mask & value) != '0`. Полная таблица — в LRM. `++`, `--`, `+=` и присваивания в выражениях имеют побочные эффекты; в RTL их удобнее оставлять в отдельных операторах, не полагаясь на порядок вычисления разных аргументов вызова.

Упражнение 06

Почему `8'h80 >>> 1` и `8'h80 >>> 1` дают разные биты? Спроектируйте сумму трёх 8-битных беззнаковых чисел без потери переноса и назовите нужную ширину.

07. typedef, enum, структуры и объединения

Пользовательский тип делает намерение частью интерфейса. Вместо передачи безымянного набора битов можно передавать команду с полями адреса, данных и признака записи. При этом packed-представление остаётся конечным вектором для аппаратуры.

typedef и перечисления

typedef вводит имя типа, а не создаёт переменную. Именованные определения лучше переиспользовать из пакета: повторно написанные похожие агрегаты не всегда имеют ту же идентичность типов.

```
package command_pkg;
  typedef logic [15:0] word_t;
  typedef enum logic [1:0] {
    IDLE = 2'b00, READ = 2'b01, WRITE = 2'b10, ERROR = 2'b11
  } operation_t;
  typedef struct packed {
    operation_t op;
    logic [7:0] address;
    word_t data;
  } command_t;
endpackage
```

Базовый тип enum задаёт ширину, знаковую и поддержку X/Z. Без явного базового типа используется int. Если метки не получили значения, первая начинается с нуля, следующие увеличиваются. Значения должны помещаться в базовом типе и удовлетворять правилам уникальности. Диапазонные формы имён генерируют несколько меток; для протоколов явная запись значений обычно нагляднее.

Enum даёт проверку совместимости при присваивании. Произвольный вектор нельзя считать допустимым состоянием только потому, что его ширина совпала. operation_t'(bits) выполняет статическое приведение и не проверяет членство. \$cast(op, bits) может проверить допустимость; в функции его возвращаемое значение надо обработать.

Методы first(), last(), next(), prev(), num(), name() помогают обходить перечисление и печатать символические имена. next() не является универсальным описанием желаемого перехода конечного автомата: переход зависит от протокола, а не от расположения меток.

Структуры и assignment patterns

В struct packed поля образуют непрерывный вектор; первое объявленное поле занимает старшие биты. Все поля должны быть допустимыми packed-элементами. Для структуры выше \$bits(command_t) равен 26. По умолчанию packed-структура unsigned; signed у всего агрегата меняет интерпретацию целого вектора.

Фрагмент внутри модуля с импортом пакета:

```
import command_pkg::*;
command_t cmd;
always_comb begin
  cmd = '{op: IDLE, address: '0, data: '0};
end
```

'{...} — assignment pattern, учитывающий тип и поля приёмника. {...} — конкатенация, собирающая элементы по своим правилам. Смешивать их по внешнему сходству нельзя. '{default: '0} удобно для заполнения подходящего агрегата, но строгие поля enum могут потребовать явной допустимой метки.

У распакованной struct поля могут иметь разнородные типы, например string и динамический массив. Она не обязана быть одним аппаратным вектором. Присваивание структуры копирует значения её полей;

если поле является handle объекта, копируется handle, а не весь объект. Структуры передаются через порты и аргументы подпрограмм с учётом совместимости типов.

Объединения

union предоставляет несколько представлений одного хранилища. В обычном union packed члены имеют одинаковую битовую ширину:

```
typedef union packed {  
    logic [31:0] word;  
    logic [3:0][7:0] bytes;  
} word_view_t;
```

Запись `u.word = 32'h11223344`; означает, что `u.bytes[3]` — `8'h11` при указанных диапазонах. Это определяется порядком packed-битов, а не endian процессора, запускающего симулятор. Распакованные объединения имеют другие правила хранения и проверки типов; их не следует применять как переносимый RTL.

union tagged хранит сведения о выбранном варианте. Конструктор tagged Some value задаёт вариант, а сопоставление с образцом через matches позволяет проверить тег и связать полезную нагрузку с локальным именем. Такой тип удобен для моделей вариантов сообщения: «данные» или «ошибка». Чтение неактивного варианта ограничено правилами языка. Синтез и поддержку pattern matching нужно проверять отдельно.

В 2023 появился union soft packed, допускающий разные размеры членов; это специальное дополнение с правилами перекрытия битов, а не разрешение разноразмерных членов в прежнем обычном packed union. Подробности совместимости — [глава 33](#).

Упражнение 07

Составьте packed-структуру: valid — 1 бит, адрес — 12 бит, данные — 32 бита. Укажите \$bits и позиции поля адреса. Почему статическое приведение enum не заменяет проверку входной команды?

08. Массивы, очереди и методы

Packed-размерности записываются **до имени**, unpacked — после. Это не просто разный стиль индексации: packed-массив является единым интегральным значением, а unpacked-массив — набором самостоятельных элементов.

Фиксированные массивы

```
logic [3:0][7:0] packed_word; // Один 32-битный объект.  
logic [7:0] memory [0:255]; // 256 отдельных восьмибитных элементов.  
logic [7:0] image [0:15][0:31];
```

packed_word[3] выбирает старший байт, memory[3] — четвёртый элемент памяти. Полный packed-вектор участвует в арифметике. Над всей unpacked-памятью оператор + не превращается в поэлементное сложение. Для этого нужен цикл.

Фрагмент процедурного кода:

```
foreach (image[row, col])  
    image[row][col] = '0;
```

В RTL такой цикл описывает требуемую логику; он не растягивает очистку на 512 тактов. Полный сброс большой памяти может помешать выводу RAM-блока. Границы в фиксированном объявлении постоянны при элаборации; индекс чтения или записи может быть вычисляемым сигналом.

Assignment pattern '{default: '0}' инициализирует совместимый массив. Присваивание целого массива копирует элементы согласно правилам совместимости типов, формы и размеров. Срез unpacked-массива сохраняет массивную природу; он отличается от битового part-select packed-вектора. Конкатенация unpacked массивов может соединять элементы и массивы подходящего типа.

Динамический массив

```
// Фрагмент тестбенча внутри initial.  
int samples[];  
samples = new[4];  
samples[0] = 12;  
samples = new[8](samples);  
samples.delete();
```

new[8](samples) создаёт новое хранилище и копирует помещающуюся часть прежнего. Без аргумента копирования старые данные не сохраняются таким способом. size() сообщает текущую длину, delete() освобождает элементы. Это программная модель памяти тестбенча, не переносимый RTL.

Очередь и ассоциативный массив

```
// Объявления и операции в тестбенче.  
int pending[$];  
int counters[string];  
// pending.push_back(17);  
// value = pending.pop_front();  
// counters["read"]++;
```

Очередь имеет последовательные индексы, изменяет длину, поддерживает push_front, push_back, pop_front, pop_back, insert, delete, size. [\$:N] ограничивает верхний индекс, то есть вместимость равна N+1. До удаления элемента проверяйте непустоту: диагностическое поведение пустого pop не является полезным протоколом. Присваивание очереди копирует элементы, но handles внутри остаются ссылками на прежние объекты.

Ассоциативный массив выделяет только использованные ключи. Тип индекса задаётся в скобках, например [string]. Методы exists, num, first, last, next, prev, delete поддерживают поиск и обход. Отсутствующий ключ и значение, равное нулю, — разные состояния; используйте exists, если это существенно.

Методы и запросы размеров

Группа	Средства	Что учитывать
Геометрия	<code>\$size</code> , <code>\$left</code> , <code>\$right</code> , <code>\$low</code> , <code>\$high</code> , <code>\$increment</code>	Номер размерности существенен
Размерности	<code>\$dimensions</code> , <code>\$unpacked_dimensions</code>	Packed и unpacked считаются по правилам запроса
Биты	<code>\$bits</code>	Размер представления, а не число элементов вообще
Поиск	<code>find</code> , <code>find_index</code> , <code>find_first</code> , <code>find_last</code> и варианты <code>_index</code>	Результат обычно очередь
Экстремумы	<code>min</code> , <code>max</code> , <code>unique</code> , <code>unique_index</code>	Это методы массива, не SVA/RTL-квалификаторы
Порядок	<code>sort</code> , <code>rsort</code> , <code>reverse</code> , <code>shuffle</code>	Меняют коллекцию
Свёртки	<code>sum</code> , <code>product</code> , <code>and</code> , <code>or</code> , <code>xor</code>	Следить за размером промежуточного значения

Выражение `values.sum() with (int'(item))` расширяет каждый элемент до `int` до суммирования. Без этого сумма узких элементов может иметь неожиданно узкий тип. `find() with (item > 10)` выбирает элементы по условию. `item` — имя итератора; в синтаксисе методов можно задавать собственное имя. В 2023 добавлен `map` для преобразования элементов; поддержку режима языка проверяют отдельно.

Массивы передают подпрограммам по значению или по ссылке. Open array argument с `[]` позволяет принять различные размеры unpacked-массива при совместимом типе элементов. Для `ref` требования строже; правила — в [главе 12](#).

Упражнение 08

Чем отличаются `$bits(logic [3:0][7:0])` и количество элементов в `logic [7:0] mem[0:3]`? Как хранить ожидаемые ответы DUT, если их количество меняется в ходе симуляции?

09. Совместимость и приведение типов

Совпадение битовых размеров ещё не означает совместимости типов. Язык различает идентичные, эквивалентные, совместимые по присваиванию и совместимые по приведению типы. Особенно заметны различия у enum, распакованных агрегатов, классов и ref.

Проверять интерфейс, а не только биты

Для обычных интегральных присваиваний возможны расширение и усечение. При присваивании enum действует более строгая типизация. При передаче ref подпрограмма получает доступ к исходному объекту, поэтому компилятор не может просто создать временную конвертированную копию, как для обычного input.

Совместимость сетей учитывает вид и тип данных, а для пользовательских nettype существенны и правила разрешения. Совместимость класса связана с наследованием; одинаковые поля двух несвязанных классов не делают handles взаимозаменяемыми.

\$typename(expr) помогает при диагностике, но строковое имя типа не следует использовать как переносимый ключ для определения его семантики. type(expr) получает тип выражения там, где синтаксис ожидает тип, например при объявлении типового параметра. Это не вычисление значения объекта.

Статические приведения

```
typedef logic signed [15:0] sample_t;
sample_t sample;
logic [7:0] raw;
// Фрагменты выражений:
// sample = sample_t'($signed(raw));
// count = int'(short_value);
// masked = 8'(wide_value);
```

T'(expr) преобразует к типу T. N'(expr) задаёт размер, сохраняя правила знаковости исходного выражения. signed'(expr) и unsigned'(expr) меняют знаковую трактовку. \$signed/\$unsigned выполняют близкую задачу для интегральных выражений без самостоятельного расширения ширины.

В приведении sample_t'(\$signed(raw)) сначала 8-битный raw трактуется как signed, затем расширяется до signed 16 бит. Если сразу привести unsigned raw к более широкому типу, отрицательное значение не появится из намерения автора.

Обычное преобразование real к целому с помощью интегрального приведения округляет к ближайшему целому, половины — от нуля. \$rtoi отсекает дробную часть к нулю. \$realtobits и \$bitstoreal переинтерпретируют битовое представление real, а не превращают числовое 1.0 в целочисленное 1. Для shortreal существуют \$shortrealtobits и \$bitstoshortreal.

Динамический \$cast

Полная единица тестбенча:

```
module enum_cast_demo;
  typedef enum logic [1:0] {STOP=0, GO=1} mode_t;
  mode_t mode = STOP;
  logic [1:0] bits = 2;
  initial begin
    if (!$cast(mode, bits))
      $display("Rejected mode; previous value is %s", mode.name());
  end
endmodule
```

При неудаче функциональная форма `$cast` возвращает 0 и не меняет назначение. Статическая запись `mode_t'(bits)` не выполняла бы эту проверку членства `enum`. Для классов `$cast` проверяет, можно ли `handle` базового класса безопасно рассматривать как `handle` нужного производного класса. Проверка не создаёт объект.

Битовые потоки и упаковка

Bit-stream `cast` преобразует допустимые агрегаты через последовательность битов. Для фиксированных представлений нужны согласованные размеры; для динамических приёмников действуют дополнительные правила определения размера. Это полезно для сериализации тестовых пакетов, но не заменяет описание формата протокола.

Определите порядок полей, индексов и байтов до написания упаковки. Packed-структура имеет заданный порядок битов; динамическая структура C по DPI может иметь `padding` и иную ABI. Нельзя объявлять их совместимыми по внешнему сходству объявлений. Поточковые операторы позволяют явно задать направление и размер блока перестановки.

Типичные ошибки

Частые причины дефекта: `cast` добавлен после переполнения; `unsigned part-select` используется как отрицательное число; результат `$cast` игнорируется; X теряется при преобразовании в `bit`; `enum` с незаконным кодом воспринимается как проверенный. Приведение должно объяснять намерение, а не просто подавлять диагностику компилятора.

Упражнение 09

При `raw=8'hFE` получите signed 16-битное -2. Чем отличаются числовое преобразование `int'(1.8)` и `$rtoi(1.8)`?

10. Присваивания и процессы RTL

Присваивание описывает либо постоянно действующий драйвер, либо действие процесса. Из выбора оператора и событийного управления складывается модель комбинационной логики, защёлки или триггера. Имя типа `logic` само по себе этого выбора не делает.

Непрерывное присваивание

```
module mux2 (
    input wire a, b, select,
    output wire y
);
    assign y = select ? b : a;
endmodule
```

`assign` — элемент модуля, а не строка внутри `always`. Правая часть пересчитывается при изменении зависимостей. Непрерывное присваивание переменной также допускается правилами SystemVerilog с ограничениями на драйверы; это не разрешение одновременно записывать тот же объект из независимых процессов.

Blocking и nonblocking

`=` обновляет левую часть в ходе выполнения оператора. Следующая строка того же последовательного процесса видит обновлённое значение. `<=` вычисляет правую часть сейчас, а обновление планирует в NBA. Блокирующий оператор не блокирует всю симуляцию: слово относится к продолжению конкретного процесса.

Обычное правило RTL: комбинационные вычисления — `=`, состояния на фронте — `<=`. Временные локальные вычисления в тактовом процессе возможны и с `=`, если они имеют ясный смысл и не становятся разделяемым состоянием. В одном сигнале смешивать `blocking` и `nonblocking` не следует.

`always_comb`, `always_latch`, `always_ff`

```
// Фрагмент модуля: next_value и q имеют одинаковый тип.
always_comb begin
    next_value = q;
    if (load) next_value = data;
    else if (step) next_value = q + 1'b1;
end

always_ff @(posedge clk)
    if (reset) q <= '0;
    else      q <= next_value;
```

`always_comb` запускается один раз в начале моделирования, учитывает зависимости читаемых выражений и вызываемых функций и запрещает временное ожидание. Переменные, которые он записывает, не должны записываться другими процессами. У старого `always @*` есть отличия в правилах чувствительности и начального запуска.

В `always_comb` не стоит читать временную переменную до её вычисления: `c = b; b = a;` может оставить `c` равным прежнему `b`. Чувствительность не делает блок итерационным решателем: значения, записываемые внутри блока, имеют особые правила исключения из неявной чувствительности. Сначала вычисляйте промежуточные значения, затем используйте их.

`always_latch` выражает намерение получить прозрачную защёлку: при закрытом разрешении состояние сохраняется. `always_ff` выражает тактовое состояние, допускает одно событийное управление и

ограничивает дополнительные временные управления и записи из других процессов. Список событий в одном `@( ... )` может содержать такт и асинхронный сброс.

Обычный `always` остаётся универсальным процессом. `always` без событий/задержек или другого блокирования может создать бесконечный цикл нулевого времени. `initial` выполняется один раз, `final` — при завершении симуляции и не должен потреблять модельное время.

Неполные присваивания

В комбинационном блоке каждый выход должен получить значение на любом пути. Иначе модель сохраняет прежнее значение, что означает память. Значения по умолчанию в начале блока помогают показать полный контракт. Присваивание `X` как `default` в синтезе может рассматриваться как `don't-care` и дать непредсказуемое поведение для фактически встретившегося кода; для управляющих выходов лучше безопасное значение.

Задержки внутри и перед присваиванием

`#5 a = b;` ждёт, затем читает `b`. `a = #5 b;` читает `b` сейчас, блокирует процесс и обновляет позже. `a <= #5 b;` читает сейчас, планирует обновление и позволяет процессу продолжиться. Эти формы служат моделированию; они не задают пятибитный или пятиступенчатый аппаратный буфер.

Наследуемые процедурные `assign/deassign` и `force/release` имеют специальные правила переопределения значений. `force` полезен при диагностике тестбенча, но способен скрыть реальный драйвер. После `release` переменная и сеть возвращаются к обычному поведению по разным правилам: сеть пересчитывается по драйверам, переменная может ждать следующей процедурной записи. Это не переносимый RTL.

Упражнение 10

Какую память опишет `always_comb if (enable) q = d;`? Исправьте описание, если требовался мультиплексор с нулём на выходе при `enable=0`.

11. Условия, выбор и циклы

Процедурные операторы выбирают действия внутри процесса. В RTL они описывают логику выбора и повторяющуюся структуру вычисления. Последовательность строк в `begin ... end` не означает последовательность тактов.

if и case

`if (cond)` выполняет ветвь при истинном условии. Нулевое или неопределённое булево значение не выбирает эту ветвь; иначе выполняется `else`, если он есть. Цепочка `if/else if` задаёт приоритет: первая выполненная проверка завершает выбор.

Обычный `case` сравнивает значения с учётом X/Z по правилам точного сопоставления. `casez` допускает Z/? как wildcard, `casex` — ещё и X. Маскирование неизвестности на входе способно скрыть дефект сброса, поэтому `casex` в управляющем RTL опасен. `case (...)` inside позволяет задавать диапазоны и наборы с асимметричными правилами wildcard сопоставления.

```
// Фрагмент комбинационного процесса.
always_comb begin
    result = '0;
    case (opcode)
        2'b00: result = a & b;
        2'b01: result = a | b;
        2'b10: result = a ^ b;
        default: result = '0;
    endcase
end
```

Наличие `default` явно задаёт поведение для прочих значений. Оно не обнаруживает ошибочный код само по себе; для этого нужна отдельная проверка входного контракта.

unique, unique0, priority

Квалификатор	Проверяемое намерение
<code>unique</code>	Не более одной подходящей ветви; без <code>default/else</code> ожидается совпадение
<code>unique0</code>	Не более одной подходящей ветви; отсутствие совпадения разрешено
<code>priority</code>	Порядок ветвей существенен; без <code>default/else</code> ожидается совпадение

Они могут порождать диагностику симулятора и давать синтезатору основания для оптимизации. Это обещание автора, которое должно быть правдой. Если оставшиеся входные комбинации на практике возникают, оптимизированная схема не обязана повторять удобное поведение исходной неполной модели.

Исторические директивы/комментарии `full_case`, `parallel_case` задают подобные подсказки отдельным синтезаторам, но обычный симулятор может их не учитывать. Получается риск расхождения симуляции и схемы. Явный полный код, стандартные квалификаторы и `assertions` лучше делают контракт проверяемым.

Циклы

`for` содержит инициализацию, условие и шаг. `repeat(N)` выполняет тело N раз. `foreach` обходит индексы массива. `while` проверяет условие до тела, `do ... while` — после, `forever` повторяет без условия остановки.

```
// Фрагмент комбинационного суммирования фиксированных элементов.
always_comb begin
    total = '0;
    for (int i = 0; i < 4; i++)
        total = total + extended_input[i];
end
```

Такой цикл описывает вычисление суммы в одном комбинационном интервале. Для суммирования за четыре такта нужны индекс-регистр и тактовый процесс. Синтезатор должен суметь ограничить/развернуть цикл; бесконечный `while` без временного управления не становится автоматически конечным автоматом.

`break` завершает ближайший цикл, `continue` переходит к следующей итерации. `return` завершает подпрограмму, а в функции может вернуть значение. `disable` именованного блока имеет более широкие исторические применения; использование для управления несколькими активациями требует осторожности.

`begin : name ... end : name` создаёт именованную область. Имена полезны для диагностики и иерархических ссылок. Объявления локальных переменных размещаются в начале соответствующего блока до исполняемых операторов.

Упражнение 11

У дешифратора две маски совпадают с одним входом. Как ведут себя обычный `case`, `unique case` и `priority case`? Почему `unique` не исправляет ошибку масок?

12. Функции, задачи и время жизни

Функция вычисляет результат без потребления модельного времени. Задача описывает процедуру, которая может ждать события или задержки. Это различие относится к семантике симуляции, а не к скорости физического вычисления после синтеза.

Функции

```
function automatic logic [7:0] saturating_inc(input logic [7:0] value);
    if (&value) return value;
    return value + 8'd1;
endfunction
```

Это объявление можно поместить в модуль или пакет. Вызов в комбинационном выражении описывает проверку максимума и инкремент. Функция не может выполнять блокирующее временное управление; косвенные вызовы также должны соблюдать ограничения функций. `function void` не возвращает полезного значения, но сохраняет ограничения на время.

Результат можно присвоить имени функции или вернуть через `return`. Явные типы аргументов и результата предотвращают случайные однобитные/неверно унаследованные объявления. Функция может иметь `output`, `inout`, `ref`, но побочные эффекты затрудняют использование как чистого выражения.

Задачи и аргументы

```
// Фрагмент тестбенча; clk, valid, payload объявлены в модуле.
task automatic drive_word(input logic [7:0] value);
    @(negedge clk);
    valid = 1'b1;
    payload = value;
    @(negedge clk);
    valid = 1'b0;
endtask
```

`input` получает значение при входе, `output` передаёт результат при выходе, `inout` копирует при входе и выходе. `ref` связывает формальный аргумент с исходной переменной: изменения видны во время выполнения. `const ref` разрешает чтение без копирования, запрещая изменение через этот аргумент. Ссылочные аргументы требуют автоматического времени жизни подпрограммы и совместимого типа реального объекта; выражение вроде `a+b` не заменяет переменную.

Можно задавать значения аргументов по умолчанию и вызывать по имени: `configure(.timeout(20), .enable(1));`. Это облегчает развитие интерфейса, но направление `input` и тип желательно повторять, если их наследование от предыдущего аргумента может запутать читателя.

Static и automatic

У обычных подпрограмм модулей по умолчанию статическое время жизни, если контекст не задаёт другое. У методов класса — автоматическое. `automatic` создаёт отдельные аргументы и локальные переменные для каждого активного вызова. Без этого параллельные вызовы задачи могут перезаписывать общие локальные данные.

Статическая локальная переменная сохраняет значение между вызовами. Инициализация при объявлении не повторяется с каждым вызовом. Для рекурсивного вычисления обычно нужно `automatic`; даже корректная рекурсивная функция не обязательно поддерживается синтезатором при динамической глубине рекурсии.

В 2023 `ref static` позволяет явно потребовать, чтобы фактический аргумент имел статическое время жизни. Это важно для операций, чьё действие переживает обычный вызов, и не означает «сделать локальную копию статической». Режим языка и ограничения конкретного применения проверяются отдельно.

Константные функции и параметры

Константная функция вызывается при элаборации, если её аргументы и тело удовлетворяют соответствующим ограничениям. Она подходит для расчёта разрядностей и таблиц. Время симуляции, текущее значение входа и динамическая побочная работа не являются допустимыми источниками элаборационной константы.

Обычные функции/задачи не имеют модульного списка `#( . . . )` параметров вызова. Обобщение делают через параметры окружающего модуля/класса, типы пакета или разные подпрограммы. Например, статический метод параметризованного класса может обслуживать разные типы в тестбенче.

Массивы и структуры передаются обычными аргументами. Для больших тестовых массивов `const ref` экономит копирование, но требует соблюдения типов и времени жизни. Не назначайте неблокирующие обновления автоматическим локальным объектам, которые могут перестать существовать к моменту применения обновления.

Пустая подпрограмма всё равно имеет интерфейс и семантику вызова; это не заглушка аппаратной задержки. Системные `$countones`, `$onehot`, `$clog2`, `$isunknown` и другие средства часто заменяют самодельные вспомогательные функции; их справочник — [приложение В](#).

Упражнение 12

Два процесса одновременно вызывают задачу, содержащую `#10` и локальный счётчик. Почему `automatic` существенно? Когда `output` станет виден вызывающему процессу, а когда изменение видно через `ref`?

13. Пакеты, области видимости и bind

Пакет объединяет определения, которые должны быть одинаковыми в нескольких модулях: тип команды, перечисление состояний протокола, константы и чистые функции. Пакет не является экземпляром аппаратного блока и не имеет портов.

Определение и импорт

```
package bus_pkg;
  parameter int ADDR_W = 12;
  typedef logic [ADDR_W-1:0] address_t;
  typedef struct packed {
    address_t address;
    logic write;
  } request_t;
endpackage

module request_register (
  input logic clk,
  input bus_pkg::request_t request_in,
  output bus_pkg::request_t request_out
);
  always_ff @(posedge clk) request_out <= request_in;
endmodule
```

`bus_pkg::request_t` — явное квалифицированное имя. `import bus_pkg::request_t;` делает доступным отдельное имя, `import bus_pkg::*;` — допускает поиск имён пакета при обращениях. Wildcard `import` не означает текстовую вставку всего пакета. Если два пакета предоставляют одинаковое имя, разрешайте неоднозначность явно.

Объявление типа в пакете должно быть скомпилировано до использования. Если имя нужно в заголовке модуля, импорт внутри тела уже слишком поздний для такого обращения; используйте квалифицированное имя или допустимый импорт в заголовке. Пакет можно реэкспортировать через `export`, но не стоит скрывать происхождение критичных интерфейсных типов за длинной цепочкой wildcard-импортов.

Области и единицы компиляции

Есть области пакетов, модулей, интерфейсов, классов, подпрограмм и именованных блоков. Внутреннее объявление может затенить внешнее. Иерархический путь обращается к конкретному экземпляру, `::` — к содержимому типа/пакета и другим областям, где разрешён этот оператор. Точка также выбирает поле структуры или член объекта; одинаковый знак не делает эти обращения эквивалентными.

`$unit` обозначает область единицы компиляции. Какие файлы составляют одну единицу, зависит от запуска компилятора. Общая переменная на уровне файла может неожиданно исчезнуть из видимости при изменении файлового списка. Для переносимых общих определений используйте пакеты. `$root` относится к корню иерархии, а не к области пакета.

Восходящий поиск иерархических имён и ссылки на внутренние сигналы допустимы в некоторых контекстах, но делают компонент зависимым от места включения. Пакет не должен подменять интерфейс модуля обращениями к случайному `top`. Встроенный пакет `std` предоставляет, например, `process`, `mailbox`, `semaphore` и средства рандомизации.

bind и checker

`bind` добавляет проверяющий модуль/интерфейс/checker в существующий контекст без редактирования исходного RTL. Это удобно, когда одна и та же проверка нужна для всех экземпляров блока.

```
// Фрагмент проекта проверки: DUT называется request_register.
module register_check(input logic clk, input bus_pkg::request_t request_in);
    assert property (@(posedge clk) !$isunknown(request_in));
endmodule

bind request_register register_check u_check(.clk(clk), .request_in(request_in));
```

Имена в подключениях разрешаются относительно целевого контекста. Привязка к типу модуля и привязка к конкретному экземпляру дают разный охват. `checker` предоставляет специализированный контейнер проверок с собственными правилами аргументов и событий; это не обычный синтезируемый модуль хранения. Семантика `assert property` подробно вводится в [главе 27](#).

Практика организации

Храните интерфейсные типы в небольших пакетах без глобального изменяемого состояния. Для проектов полезен явный файловый список: пакеты, интерфейсы, RTL, затем тестбенч. Не включайте один и тот же файл пакета через `include` во множество модулей: это создаёт повторные определения и зависимость от `guards`. Условная компиляция пакета должна иметь одинаковый смысл во всех зависимых файлах.

Упражнение 13

Почему включение `import p::*;` в тело модуля не помогает типу `p::T`, использованному без квалификации раньше, в списке портов? Когда `bind` удобнее непосредственного экземпляра проверяющего модуля?

14. Параметры и generate

Параметризация меняет структуру до начала симуляции. Один и тот же модуль можно элаборировать с разной шириной, количеством каналов и типом данных. Изменение сигнала во время работы не может создать новый экземпляр аппаратуры.

Значения и типы параметров

```
module delay_cell #(
    parameter type T = logic [7:0]
) (
    input logic clk,
    input T d,
    output T q
);
    always_ff @(posedge clk) q <= d;
endmodule
```

Экземпляр с `#( .T(logic [15:0]))` хранит 16 бит. Параметр типа сохраняет структуру интерфейса лучше, чем отдельные независимые числа ширины полей. Значение `parameter` переопределяют в экземпляре по имени; производные размеры вычисляют через `localparam`, чтобы пользователь не мог нарушить инвариант.

`$clog2(N)` вычисляет округление логарифма вверх. Для $N=1$ результат равен 0, но `logic [-1:0]` не является вектором нулевой длины. Поэтому ширину указателя часто задают как $(N > 1) ? \$clog2(N) : 1$. Для счётчика $0..N$ включительно нужно $\$clog2(N+1)$ бит с учётом нижней границы. Ограничения $N > 0$ проверяйте отдельно; отрицательные/нулевые параметры не должны молча порождать странные диапазоны.

Циклическая генерация

```
module lane_registers #(parameter int N = 4) (
    input logic clk,
    input logic [N-1:0] d,
    output logic [N-1:0] q
);
    for (genvar i = 0; i < N; i++) begin : g_lane
        always_ff @(posedge clk) q[i] <= d[i];
    end
endmodule
```

`genvar` участвует в элаборации; во время симуляции здесь уже есть N процессов. Для каждой итерации создаётся отдельная область, доступная как `g_lane[0]`, `g_lane[1]` и так далее. Индекс генерации доступен как локальная константа соответствующей итерации. Явные имена защищают тестбенч от нестабильных `genblkN`.

Ключевые слова `generate/endgenerate` могут группировать конструкции; в контекстах SystemVerilog, где генерация однозначна, они не обязательны. Не путайте это с процедурным `for`: последний выполняется при активации процесса, хотя синтезатор тоже может развернуть его вычисление в схему.

Условная генерация

`if` или `case` вне процедурного блока с постоянным условием выбирает ветвь структуры. Например, параметр `USE_PIPELINE` выбирает вариант с регистром или прямой передачей. Только выбранная ветвь становится частью элаборированной иерархии. Но обе ветви должны удовлетворять синтаксису исходного языка.

В процедурном `if (enable)` сигнал выбирает действие уже существующей схемы. В `generate if (HAS_PARITY)` параметр определяет, существует ли схема чётности. Макрос ``ifdef` действует ещё раньше — на текст; типовая проверка исключённого текста вообще может не происходить.

Валидация параметров

Проверяйте не только значения по умолчанию. Обязательные случаи: минимальная ширина, единичная глубина, неполная степень двойки, максимально поддерживаемый размер. Производные диапазоны не должны переполнять тип параметра. `Runtime $fatal` в `initial` даёт понятное сообщение симуляции, но не заменяет гарантированную диагностику синтезатора: специализированные проверки параметров зависят от `flow`.

`defparam` — старый способ изменения параметров по иерархическому имени; его зависимость от структуры делает сопровождение труднее. Для нового RTL предпочтительны явные `#(.NAME(value))` на месте создания экземпляра.

Упражнение 14

Определите ширину адреса и ширину счётчика заполнения FIFO глубины 1, 5 и 16. Почему при глубине 5 недостаточно полагаться на естественное переполнение указателя?

15. Интерфейсы и modport

Интерфейс объединяет связанные сигналы в именованный объект и может описывать допустимые роли участников. Вместо независимых портов `valid`, `ready`, `data` модуль получает один интерфейс потока. Это уменьшает вероятность перепутать каналы, но само по себе не проверяет временной протокол.

Объявление и роли

```
interface stream_if #(parameter int W = 8) (input logic clk);
    logic valid, ready;
    logic [W-1:0] data;
    modport source (input clk, ready, output valid, data);
    modport sink   (input clk, valid, data, output ready);
endinterface

module stream_wire(stream_if.sink input_bus, stream_if.source output_bus);
    assign output_bus.valid = input_bus.valid;
    assign output_bus.data = input_bus.data;
    assign input_bus.ready = output_bus.ready;
endmodule
```

Направления `modport` читаются **со стороны подключённого модуля**. Источник выдаёт `valid/data` и принимает `ready`. Приёмник делает обратное. `Modport` не создаёт отдельную копию сигналов и не является механизмом аппаратной защиты. Он задаёт допустимый доступ через соответствующий вид интерфейса.

Экземпляры и соединения

В верхнем модуле создают `stream_if #(.W(8)) bus(clk);`, затем передают `bus` в интерфейсный порт экземпляра. Можно указать роль в объявлении порта модуля или выбрать разрешённый `modport` при подключении. Согласуйте эти способы, чтобы не получить конфликтующие объявления ролей.

Типизированный порт `stream_if.sink` фиксирует семейство интерфейса. Generic-форма с ключевым словом `interface` оставляет конкретный интерфейс для соединения; она полезна обобщённым блокам, но уменьшает очевидность контракта. Параметры экземпляра интерфейса должны давать согласованные ширины для обеих сторон.

Интерфейсы могут передаваться другим интерфейсам; возможны массивы экземпляров. Объявления на уровне проекта удобнее локально вложенных вариантов для совместного использования. `.name` и `.*` сохраняют общую идею неявных соединений, однако совместимость интерфейсов и `modport` проверяется по отдельным правилам.

Методы и внутренние процессы

Интерфейс может содержать функции, задачи, непрерывные присваивания и процессы. `import` в `modport` предоставляет доступ к методу интерфейса; `export` участвует в схемах, где реализацию метода предоставляет подключённая сторона. Это не тот же `import`, что импорт пакета, и не `DPI-import`.

Хранить счётчик протокола внутри интерфейса допустимо, но нужно понимать, кто им владеет и сколько экземпляров создаётся. Не переносите в общий интерфейс скрытые драйверы сигналов, которые уже записывает модуль-источник. Многие синтезаторы поддерживают только определённое подмножество интерфейсных возможностей; наличие задачи с `@` меняет характер интерфейса на проверяющий.

Clocking и virtual interface

Clocking block задаёт момент наблюдения и подачи сигналов тестбенча относительно события такта. Он имеет собственные направления и skew; это отличается от modport, который в первую очередь описывает доступ. Подробнее — [глава 23](#).

Класс не создаёт аппаратный экземпляр интерфейса. `virtual interface` хранит ссылку на уже существующий интерфейс, чтобы драйвер или монитор класса мог работать с сигналами. До доступа такую ссылку нужно присвоить; `null` не является интерфейсом. Параметры и выбранный `modport` участвуют в совместимости `virtual`-типа.

Для переносимого RTL начинайте с простого интерфейса данных и `modport`. Временные задачи, `clocking`, проверки и `virtual handles` добавляйте в слой проверки с ясным разделением владельцев сигналов.

Упражнение 15

Запишите роли источника и приёмника для `request`, `acknowledge`, `address`. Какой стороне принадлежит `acknowledge`? Почему два экземпляра интерфейса с одинаковыми именами полей всё равно являются разными соединениями?

16. Препроцессор и единицы компиляции

Препроцессор преобразует текст до разбора типов и элаборации. Макрос не знает, является ли его аргумент сигналом или выражением с побочным эффектом. Поэтому функции, параметры и пакеты обычно надёжнее для семантического повторного использования.

include и define

```
`ifndef PROJECT_DEFS_SVH
`define PROJECT_DEFS_SVH
`define PROJECT_MAX_CHANNELS 8
`define CLAMP_HIGH(x, limit) (((x) > (limit)) ? (limit) : (x))
`endif
```

Это содержимое заголовочного .svh. Guard предотвращает повторное включение в одной препроцессорной области, но не управляет глобально всеми отдельными запусками компилятора.

``include "file.svh"` вставляет текст; пути поиска задаются инструментом. Пакет с типами обычно компилируют отдельным .sv.

Аргументы макросов подставляются текстом. В примере `x` встречается дважды; передача `i++` может дать неожиданные действия. Скобки защищают приоритет, но не устраняют повторное вычисление. Многострочное определение продолжают обратной косой чертой перед переводом строки.

Склейка и строки

Двойной обратный апостроф склеивает части токена, например при построении имени сигнала.

Специальное макро-кавычkovание позволяет включить раскрытые аргументы в строку. Это инструменты для генерации повторяющегося текста, а не система типов; сложные ошибки часто проще понять по выводу препроцессора.

```
`define REG_NAME(n) reg_`n
`define AS_TEXT(x) `"x"
// `REG_NAME(3) становится reg_3; `AS_TEXT(hello) – строкой "hello".
```

`__FILE__` и `__LINE__` раскрывают имя файла и номер строки. Директива ``line` изменяет отображаемые позиции для сгенерированного кода. В этой методичке обратные апострофы в исходниках всегда буквальны, не форматирование оболочки команд.

Условная компиляция

``ifdef`, ``ifndef`, ``elsif`, ``else`, ``endif` выбирают текст по наличию определений. `define FEATURE 0` всё равно определяет `FEATURE`; обычный `ifdef FEATURE` не вычисляет арифметическое значение нуля. `undef` удаляет одно определение, `undefineall` — определения в соответствующем препроцессорном состоянии. В 2023 расширена запись логических условий директив; для старых инструментов используйте традиционные вложенные проверки.

Один и тот же набор `defines` должен применяться согласованно к пакету типов, RTL и тестбенчу. Если ширина структуры изменена только в одном файле, компилируемый проект может получить несовместимые интерфейсы.

Директивы окружения

Директива	Назначение
<code>default_nettype</code>	Управляет неявными сетями; none обнаруживает опечатки
<code>timescale</code>	Единица времени и точность для последующих описаний
<code>resetall</code>	Возвращает настройки директив к исходным значениям; не заменяет <code>undefineall</code>
<code>unconnected_drive</code> , <code>nounconnected_drive</code>	Поведение неподключённых входов в соответствующей области
<code>celldefine</code> , <code>endcelldefine</code>	Помечают библиотечные ячейки
<code>begin_keywords</code> , <code>end_keywords</code>	Выбирают набор ключевых слов редакции языка
<code>pragma</code>	Стандартизованная оболочка и инструментальные расширения

В новом RTL полезно начинать файл с ``default_nettype none` и явно восстанавливать нужный контекст на границе проекта, если рядом сторонние исходники. Не рассчитывайте на случайную директиву из ранее скомпилированного файла. В 2023 ``timescale` внутри `design element` запрещена; объявления `timeunit` и `timeprecision` делают локальное намерение точнее.

Упражнение 16

Почему ``ifdef FEATURE` выбирает ветвь после ``define FEATURE 0`? Что лучше для ширины экземпляра: глобальный макрос или параметр модуля, и почему?

17. Комбинационная логика и арифметика

Комбинационный блок не хранит состояние: при одинаковых входах его установившийся выход одинаков независимо от истории. В RTL это выражается непрерывными присваиваниями или полными `always_comb`. Полнота означает значение на каждом пути, а не просто наличие слова `default` где-то в коде.

Дешифраторы и мультиплексоры

```
module decode4(input logic [1:0] select, output logic [3:0] one_hot);
  always_comb begin
    one_hot = '0;
    case (select)
      2'd0: one_hot[0] = 1'b1;
      2'd1: one_hot[1] = 1'b1;
      2'd2: one_hot[2] = 1'b1;
      2'd3: one_hot[3] = 1'b1;
      default: one_hot = '0;
    endcase
  end
endmodule
```

Для известных двоичных входов ровно один выход активен. Для X/Z выбран безопасный нулевой результат, но отдельная `assertion` должна сообщить о нарушении контракта, если неизвестные входы запрещены. Без неё безопасное значение может скрывать ошибку.

Переменный индекс в присваивании вектора может синтезироваться в дешифратор, а чтение `values[index]` — в мультиплексор. Цена зависит от количества входов и их ширины. Конструкция короче в тексте не обязательно дешевле в схеме.

Приоритет и длина пути

Цепочка `if/else if` описывает приоритет. Параллельный выбор по взаимно исключающимся кодам может позволить более плоскую реализацию. Синтезатор оптимизирует логику, но не должен менять наблюдаемую семантику для допустимых входов. Квалификаторы `unique` и `priority` сообщают дополнительные обещания, которые проверяются отдельно.

Комбинационная петля возникает, если выход зависит от самого себя без хранения по предусмотренному элементу. `assign a = ~a;` — не генератор такта для обычного RTL flow; у симулятора нет устойчивого значения, а физическая реализация не следует цифровой модели с нулевыми задержками.

Арифметический контракт

До кода задайте: `signed/unsigned`, ширину входов, ширину результата, обработку переполнения и задержку в тактах. Например, сумма двух `signed` 8-битных величин лежит в диапазоне -256..254 и требует 9 бит. Беззнаковая сумма лежит в 0..510 и тоже требует 9 бит, но интерпретация старшего бита и сравнения различаются.

При насыщении результат ограничивается минимумом/максимумом; при модульной арифметике старшие биты отбрасываются. Ни один вариант не является «правильным по умолчанию» для всех протоколов. Сохраняйте расширенный промежуточный результат до принятия решения о насыщении.

Деление на константную степень двойки беззнакового числа можно выразить сдвигом. Для `signed` отрицательных чисел арифметический сдвиг и целочисленное деление могут различаться округлением: -3/2 даёт -1, а знаковый сдвиг вправо — -2. Умножение и деление на переменную могут быть дорогими; синтез не превращает их автоматически в нужный многотактовый протокол.

Фиксированная точка

Язык не назначает обычному `logic signed` положение двоичной точки. В формате с F дробными разрядами хранимое целое X означает $X/2^F$. При умножении числа дробных разрядов складываются. Прежде чем усечь результат обратно, решите, требуется ли округление, насыщение и какой допустим уровень ошибки.

Выделение `product[N:L]` без пояснения масштаба затрудняет проверку. Назовите типы и константы масштаба, а тесты включайте для отрицательных чисел, нуля, минимума, максимума и границы округления.

Упражнение 17

Разработайте 8-битный беззнаковый сумматор с насыщением: если сумма больше 255, выдавать 255. Какой промежуточный размер нужен и какой бит определяет насыщение?

18. Регистры, сброс и конвейеры

Последовательностная схема хранит состояние. В синхронной модели состояние обновляется на выбранном фронте, а между фронтами комбинационная логика вычисляет следующее значение. Все регистры одного домена логически принимают новые значения одновременно; неблокирующие присваивания выражают эту модель.

Сброс и enable

```
module counter #(parameter int W = 8) (  
    input logic clk, rst_n, enable,  
    output logic [W-1:0] count  
);  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n) count <= '0;  
        else if (enable) count <= count + 1'b1;  
    end  
endmodule
```

Асинхронный сброс реагирует независимо от такта. Синхронный сброс проверялся бы внутри `always_ff @(posedge clk)` и действовал только на фронте. Полярность сигнала и приоритет относительно `enable` являются частью интерфейса.

Асинхронное утверждение сброса удобно для быстрого перевода в безопасное состояние, но снятие сброса должно удовлетворять требованиям `recovery/removal` и согласованию в домене. Цифровая симуляция не доказывает физическую корректность снятия сброса.

Отсутствие ветви `else` у тактового процесса означает сохранение значения. В комбинационном процессе похожая запись могла бы означать защёлку — контекст событийного управления принципиален. Присваивание `q <= q` обычно избыточно.

Конвейер

```
// Фрагмент модуля; d, stage0, stage1 — векторы одинаковой ширины.  
always_ff @(posedge clk) begin  
    stage0 <= d;  
    stage1 <= stage0;  
end
```

На одном фронте `stage1` принимает старое `stage0`, а не новое `d`. Так создаются две ступени хранения. Если отдельно идут `valid` и данные, их задержки должны совпадать. Сброс можно применять только к `valid`-регистрам, когда контракт запрещает использовать данные при `valid=0`; это уменьшает ресурсы сброса, но требует последовательного соблюдения контракта.

Latency — число тактов от принятия входа до доступности соответствующего выхода. **Throughput** — частота приёма/выдачи новых элементов в установившемся режиме. Двухступенчатый конвейер может иметь задержку два такта и пропускную способность один элемент за такт. Эти показатели не противоречат друг другу.

Загрузка с приоритетом и конфликты

В одном процессе последнее выполненное присваивание одному регистру определяет итог для соответствующего планирования. Однако запись «сначала инкремент, потом при условии загрузка» менее ясна, чем явное `if/else if`. Присваивания из разных тактовых процессов не следует использовать как способ расставить приоритет: это несколько владельцев состояния и возможная гонка.

Enable обычно синтезируется в вход CE или мультиплексор данных. Не делайте самодельный gated clock через `clk & enable`: изменение `enable` может создать ложный импульс. Для ограничения тактирования существуют библиотечные ячейки и рекомендации FPGA-производителя.

Инициализация и защёлки

Начальное значение при объявлении или `initial` может поддерживаться конкретной FPGA, но в ASIC не является универсальным описанием сброса. Укажите требуемое поведение после питания и сверяйте с целевой технологией.

Защёлка прозрачна при активном уровне разрешения и хранит значение при закрытии. Она может быть намеренным элементом специализированной архитектуры; тогда используют `always_latch`, проверяют временной режим и доступность ячейки. В учебном синхронном RTL непреднамеренная защёлка чаще означает неполное присваивание.

Упражнение 18

К двум ступеням данных добавьте `valid`-конвейер со сбросом. Почему для недействительных данных сброс всех битов может быть излишним? Какие свойства протокола делают такой выбор допустимым?

19. Память, ROM и очереди FIFO

Объявление массива задаёт хранилище в модели, но не гарантирует получение конкретного RAM-блока. Синтезатор распознаёт шаблон чтения, записи, тактирования и сброса, затем сопоставляет его с ресурсами выбранной технологии.

Простой RAM-порт

```
module sync_ram #(
    parameter int W = 8,
    parameter int DEPTH = 16,
    parameter int AW = (DEPTH > 1) ? $clog2(DEPTH) : 1
) (
    input logic clk, we,
    input logic [AW-1:0] address,
    input logic [W-1:0] write_data,
    output logic [W-1:0] read_data
);
    logic [W-1:0] memory [0:DEPTH-1];
    always_ff @(posedge clk) begin
        if (we) memory[address] <= write_data;
        read_data <= memory[address];
    end
endmodule
```

В RTL при одновременном чтении и записи того же адреса `read_data` получает старое содержимое: обе правые части вычисляются до NBA. Реальный RAM-блок может иметь режимы `read-first`, `write-first`, `no-change` и ограничения коллизий. Проверьте, какой режим распознал синтезатор, особенно для разных портов/тактов. Если `DEPTH` не степень двойки, часть двоичных адресов недопустима; контракт должен запрещать их или код обязан обработать их явно.

Асинхронное `assign read_data = memory[address];` описывает другой интерфейс чтения. На FPGA он может потребовать распределённую память или регистры, в то время как блоковая память имеет синхронный выход. Добавить регистр после синтеза без изменения протокола нельзя: это меняет задержку.

ROM и начальные данные

ROM можно описать через `case` адреса или константную таблицу. `$readmemh` и `$readmemb` загружают массив из файла в симуляции; многие FPGA-flow поддерживают их и для инициализации памяти, но пути, формат и попадание файла в проект надо проверить. В ASIC такая запись не создаёт автоматически ROM-макрос.

Сброс всех ячеек циклом в одном такте описывает массовую запись и часто мешает выводу специализированной RAM. Если нужно пометить буфер пустым, сбрасывайте указатели/счётчики/`valid`, а не обязательно каждый бит данных. Если требуется физическая очистка, определите отдельную многотактовую процедуру очистки.

FIFO как контракт

FIFO сохраняет порядок принятых элементов. Для интерфейса `valid/ready` передача происходит на фронте, когда оба сигнала равны единице. Источник удерживает `valid` и данные до принятия. Приёмник может снимать `ready`, создавая `backpressure`.

У синхронного FIFO обычно есть массив, указатель записи, указатель чтения и счётчик заполнения. `empty` означает `count=0`, `full` — `count=DEPTH`. При только записи `count` увеличивается; при только чтении уменьшается; при обеих передачах не меняется. События считаются по принятым операциям, а не по

одному сигналу желания `valid` или `ready`.

Для неполной степени двойки указатели должны явно переходить из `DEPTH-1` в `0`. Для `DEPTH=1` оба указателя постоянно равны `0`, но `count` всё ещё различает пустое и полное состояния. Эти два случая обнаруживают многие ошибки параметризации.

Варианты поведения на границе

Если FIFO полон и одновременно есть чтение, можно разрешить запись замещающего элемента в этот же такт. Можно выбрать и более простой контракт: принимать запись только когда до фронта FIFO не был полон. Второй вариант иногда теряет один такт на границе, зато не создаёт комбинационную зависимость `ready` входа от `ready` выхода.

Аналогично, пустой FIFO может иметь `bypass` (прямую передачу нового входа) или выдавать элемент только после его сохранения. Выбранный вариант нужно описать и проверить. Сквозной пример следующей группы использует без `bypass` и без записи при полном буфере, даже если одновременно идёт чтение.

Упражнение 19

Для FIFO глубины 3 выпишите значения `count` при последовательности: запись, запись, одновременные чтение/запись, чтение, чтение. Считайте все запрошенные операции допустимыми. В какие моменты FIFO пуст?

20. Конечные автоматы и протоколы

Конечный автомат хранит ограниченное состояние и выбирает переход по входам. Он удобен, когда действие требует нескольких тактов: ожидание запроса, запуск операции, ожидание завершения, выдача ответа.

Moore и Mealy

У автомата Moore выход зависит от состояния, у Mealy — также от текущих входов. Mealy может реагировать без дополнительного такта, но создаёт комбинационные пути через границу блока. Если несколько блоков зависят друг от друга комбинационно, легко получить длинный путь или петлю.

Полный небольшой автомат с однотоковым выходом done:

```
module controller (  
    input logic clk, rst_n, start, finished,  
    output logic busy, done  
);  
    typedef enum logic [1:0] {IDLE, RUN, REPORT} state_t;  
    state_t state, next_state;  
  
    always_comb begin  
        next_state = state;  
        busy = 1'b0;  
        done = 1'b0;  
        case (state)  
            IDLE: if (start) next_state = RUN;  
            RUN: begin  
                busy = 1'b1;  
                if (finished) next_state = REPORT;  
            end  
            REPORT: begin  
                done = 1'b1;  
                next_state = IDLE;  
            end  
            default: next_state = IDLE;  
        endcase  
    end  
  
    always_ff @(posedge clk or negedge rst_n)  
        if (!rst_n) state <= IDLE;  
        else state <= next_state;  
endmodule
```

done активен один полный интервал состояния REPORT. Если start остаётся высоким до возвращения в IDLE, автомат примет новый запуск. Если это не нужно, протокол должен требовать импульс, handshake или ожидание снятия start.

Разделение состояния и вычисления

Два процесса отделяют регистр состояния от функции перехода. Однопроцессный стиль тоже корректен, если временная семантика ясна. Главный критерий — понятность момента принятия входа, обновления регистров и доступности выхода.

next_state = state описывает сохранение текущего состояния по умолчанию. Выходы получают отдельные безопасные значения, чтобы избежать защёлок. Регистр данных может обновляться в самостоятельном always_ff, но каждый регистр должен иметь одного владельца.

Кодирование и ошибочные состояния

Binary-кодирование экономит триггеры, one-hot может упростить переходы. Синтезатор способен перекодировать FSM согласно своим настройкам. Явно заданный код состояния не всегда сохраняется физически без дополнительных ограничений. Не подключайте внешние потребители к внутреннему коду состояния без контракта.

`default: next_state = IDLE` задаёт поведение модели для неопisanного кода. Но безопасность при аппаратном сбое зависит от сохранения такой логики синтезом и от выбранной архитектуры, а не только от наличия `default`. Assertion допустимых состояний обнаруживает ошибку; она не восстанавливает схему.

Протоколы и liveness

Safety-свойство говорит «плохое никогда не происходит», например `done` и `busy` не активны одновременно. Liveness говорит «хорошее когда-нибудь происходит», например любой принятый запрос завершается. Для второго нужны допущения: если `finished` никогда не приходит, автомат не обязан сам завершить работу.

Тестируйте переходы, повторный запрос, сброс посреди RUN, длительные ожидания и неожиданные комбинации входов. Покрытие всех состояний не гарантирует покрытия всех переходов и последовательностей протокола.

Упражнение 20

Сформулируйте контракт для `start`: импульс или `valid/ready`? Как изменить автомат, чтобы удерживать `done` до подтверждения `ack` от потребителя?

21. Тактирование, CDC и переносимость

SystemVerilog задаёт функциональное поведение модели. Физическое выполнение требует соблюдения временных ограничений элементов: setup/hold, recovery/removal, периода такта и задержек распространения. Успешная RTL-симуляция не проверяет их автоматически.

Тактовый домен и временные ограничения

Домен — группа состояний, обновляемых согласованным тактом. Между регистрами одного домена нужно уложить комбинационный путь в доступное время. Период, входные/выходные задержки, generated clocks и исключения задаются в constraints средствами flow, часто SDC/XDC; это не часть грамматики SV.

Не добавляйте false path, чтобы просто убрать предупреждение анализа. Исключение должно соответствовать архитектуре передачи и быть обосновано. Multicycle path также требует корректной логики разрешений и отдельных условий для проверки setup/hold.

Однобитный асинхронный уровень

```
// Фрагмент принимающего домена, без обязательного здесь сброса.  
logic sync_first, sync_second;  
always_ff @(posedge destination_clk) begin  
    sync_first  <= asynchronous_level;  
    sync_second <= sync_first;  
end
```

Две ступени уменьшают вероятность распространения метастабильности дальше. Это не абсолютная гарантия; оценка надёжности зависит от технологии, частот, времени разрешения и размещения. Атрибуты синхронизаторов и правила размещения нужно брать из документации выбранного производителя.

Первую ступень обычно не используют в функциональной логике и не нагружают несколькими независимыми потребителями. Не вставляйте произвольную комбинационную логику между ступенями, рассчитывая сохранить ту же надёжность.

Импульсы и шины

Короткий импульс может полностью пройти между фронтами принимающего домена. Два триггера не гарантируют его обнаружение. Используют растяжение при обоснованных частотах, toggle-передачу или handshake с подтверждением.

Синхронизация каждого бита многобитной шины независимо не обеспечивает согласованного слова: биты могут быть захвачены в разные такты. Для медленно меняющихся данных применяют протокол удержания данных с синхронизацией признака; для потока — асинхронный FIFO с отдельной проверенной архитектурой.

Gray-код помогает передавать указатели, потому что между соседними значениями меняется один бит, но он требует ограничений задержек/перекоса путей. Нельзя механически превратить синхронный FIFO этой методички в асинхронный, заменив один такт двумя.

Reset domain crossing

Сброс имеет собственные домены и последовательность снятия. Часто применяют асинхронное утверждение и синхронизированное снятие отдельно в каждом домене. Состояния протокола должны учитывать, что соседний домен уже работает или ещё удерживается в сбросе. Без этого потеря транзакции возможна даже при корректной синхронизации одного сигнала.

Переносимость RTL

Разделяйте три уровня: конструкция разрешена LRM; компилятор её принимает; синтезатор выводит требуемый аппаратный ресурс. Поддержка интерфейсов, памяти, начальных значений и атрибутов зависит от версии инструмента и цели. Для vendor IP храните версию, параметры генерации, модель симуляции и необходимые ограничения рядом с интеграцией.

После изменения RTL проверяйте lint, элаборацию вариантов параметров, функциональные тесты, отчёт синтеза и timing. Для CDC/RDC нужен соответствующий анализ структуры и протоколов, а не только запуск случайного тестбенча. Сведения о синтезируемом подмножестве следует сверять, например, с [AMD UG901](#).

Упражнение 21

Почему два синхронизатора для двух битов двоичного счётчика не гарантируют правильного нового значения? Какой протокол применим для редкой передачи стабильного 32-битного слова между доменами?

22. Сквозной проект: синхронный FIFO

Практическая цель — соединить параметры, память, арифметику указателей, тактовое состояние и проверку порядка данных в одном небольшом устройстве. Полные исходники находятся в [examples: sync_fifo.sv](#) и [tb_fifo.sv](#).

Контракт до реализации

FIFO имеет один такт, активный низким уровнем асинхронный сброс, входные `in_valid/in_ready/in_data` и выходные `out_valid/out_ready/out_data`. Ширина `W` и глубина `DEPTH` положительны. Передача принимается на `posedge`, если `valid` и `ready` одновременно равны 1 непосредственно перед фронтом.

Источник сохраняет `in_valid` и `in_data`, пока вход не принят. FIFO сохраняет `out_valid` и `out_data` при остановке потребителя. В сбросе оба признака готовности/действительности на границе подавлены. Содержимое памяти не очищается; после сброса `count=0` и прежние данные недействительны.

Вариант намеренно простой: комбинационное чтение первого элемента, без `bypass` при пустоте. Когда FIFO полон до фронта, новая запись запрещена даже при одновременном чтении. Это часть контракта, а не пропущенная оптимизация. Для `block RAM` с синхронным чтением потребуется другая организация выхода.

Состояние и инварианты

Храним `read_ptr`, `write_ptr` и `count`. Ширина указателя — $\max(1, \lceil \log_2(DEPTH) \rceil)$, ширина `count` — $\max(1, \lceil \log_2(DEPTH+1) \rceil)$. Указатели явно оборачиваются при `DEPTH-1`.

Условие до фронта	Запись	Чтение	Новое count
Только входной handshake	Да	Нет	<code>count+1</code>
Только выходной handshake	Нет	Да	<code>count-1</code>
Оба handshake	Да	Да	<code>count</code>
Ни одного	Нет	Нет	<code>count</code>

Инварианты: $0 \leq \text{count} \leq \text{DEPTH}$; указатели находятся в диапазоне; выданная последовательность — префикс принятой последовательности; выход не меняет элемент при `out_valid && !out_ready`. Счётчик обновляется из одной таблицы действий, чтобы одновременные операции не стали двумя конфликтующими присваиваниями.

Полный RTL

Этот листинг совпадает с автономным исходником примера.

```
`default_nettype none
module sync_fifo #(
    parameter int unsigned W = 8,
    parameter int unsigned DEPTH = 4,
    localparam int unsigned PW = (DEPTH > 1) ? $clog2(DEPTH) : 1,
    localparam int unsigned CW = (DEPTH > 1) ? $clog2(DEPTH + 1) : 1
) (
    input  wire clk, rst_n,
    input  wire in_valid,
    output wire in_ready,
    input  wire [W-1:0] in_data,
    output wire out_valid,
```

```

input wire out_ready,
output wire [W-1:0] out_data
);
timeunit 1ns;
timeprecision 1ps;
logic [W-1:0] memory [0:DEPTH-1];
logic [PW-1:0] read_ptr, write_ptr;
logic [CW-1:0] count;
wire push = in_valid && in_ready;
wire pop = out_valid && out_ready;

assign in_ready = rst_n && (count < CW'(DEPTH));
assign out_valid = rst_n && (count != 0);
assign out_data = out_valid ? memory[read_ptr] : '0;

function automatic logic [PW-1:0] advance(input logic [PW-1:0] ptr);
    if (ptr == PW'(DEPTH - 1)) return '0;
    return ptr + 1'b1;
endfunction

// Memory has no reset: only accepted writes change its contents.
always_ff @(posedge clk)
    if (push) memory[write_ptr] <= in_data;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        read_ptr <= '0;
        write_ptr <= '0;
        count <= '0;
    end else begin
        if (push) write_ptr <= advance(write_ptr);
        if (pop) read_ptr <= advance(read_ptr);
        case ({push, pop})
            2'b10: count <= count + 1'b1;
            2'b01: count <= count - 1'b1;
            default: count <= count;
        endcase
    end
end

initial begin
    if (W < 1 || DEPTH < 1) $fatal(1, "W and DEPTH must be positive");
end
endmodule
`default_nettype wire

```

Как устроен тест

Самопроверяющийся тестбенч хранит очередь ожидаемых данных в простом массиве. Он меняет стимулы на `negedge`, перед следующим `posedge` вычисляет ожидаемые `handshake` и проверяет доступные выходы. После фронта даёт NBA завершиться и проверяет новое состояние интерфейса. Так тест не соперничает с DUT за изменение входов в Active одного фронта.

Есть направленные фазы: сброс, пустое чтение, заполнение до отказа, остановка потребителя, полное опустошение, одновременные передачи и сброс непустой очереди. Псевдослучайная фаза с фиксированным начальным состоянием генератора проверяет сочетания `valid/ready` и оборачивание указателей. Последующее опустошение проверяет оставшиеся элементы. Незавершённая передача источника удерживается до принятия.

Тест запускается для `DEPTH=1`, `DEPTH=3` и `DEPTH=8`; неполная степень двойки обнаруживает ошибки `wgap`, единичная глубина — ошибки нулевых диапазонов. Тайм-аут завершает зависший прогон с ошибкой, а любое несовпадение вызывает `$fatal`.

Что доказывает проверка

Компиляция проверяет синтаксис, имена, типы и часть правил драйверов. Симуляция проверяет поведение только для выполненных трасс. Она не доказывает timing, вывод RAM или все возможные входные последовательности. Фактически выполненные команды и результаты указаны в [VALIDATION.md](#); отсутствие симулятора не должно превращаться в выдуманное сообщение PASS.

Полный самопроверяющийся тестбенч

Листинг совпадает с `examples/tb/tb_fifo.sv`. Каждый экземпляр `fifo_case` независимо запускает направленные и псевдослучайные сценарии; верхний модуль ожидает успех всех трёх. Счётчики `accepted/delivered` включают и транзакции, отброшенные сбросом, поэтому их итоговое равенство не требуется. Проверяется опустошение эталонной очереди после последней фазы.

```
`default_nettype none
module fifo_case #(parameter int W = 8, DEPTH = 3) (output logic done = 0);
    timeunit 1ns;
    timeprecision 1ps;
    logic clk = 0;
    logic rst_n = 0, in_valid = 0, out_ready = 0;
    logic [W-1:0] in_data = '0;
    wire in_ready, out_valid;
    wire [W-1:0] out_data;
    sync_fifo #(W(W), .DEPTH(DEPTH)) dut(.);
    always #5ns clk = ~clk;

    // Independent sequence model: no DUT pointer/wrap arithmetic.
    logic [W-1:0] expected [0:8191];
    int head = 0, tail = 0;
    int accepted = 0, delivered = 0, simultaneous = 0, blocked = 0;
    bit holding = 0;
    logic [W-1:0] held_data;
    bit source_pending = 0;
    logic [W-1:0] source_data;
    bit last_push = 0;
    int unsigned rng = 32'hC0DE_1234 + DEPTH;

    function automatic int unsigned next_random();
        rng ^= rng << 13;
        rng ^= rng >> 17;
        rng ^= rng << 5;
        return rng;
    endfunction

    task automatic check_interface();
        if (in_ready !== (rst_n && (tail - head < DEPTH)))
            $fatal(1, "DEPTH=%0d in_ready mismatch", DEPTH);
        if (out_valid !== (rst_n && (tail != head)))
            $fatal(1, "DEPTH=%0d out_valid mismatch", DEPTH);
        if (out_valid && out_data !== expected[head])
            $fatal(1, "DEPTH=%0d expected=%h got=%h", DEPTH, expected[head], out_data);
    endtask

    task automatic cycle(input bit want_push, want_pop,
                        input logic [W-1:0] value);
        bit take_input, take_output;
        @(negedge clk);
        // Keep an unaccepted source transaction stable, regardless of new wishes.
        if (!source_pending && want_push) begin
            source_pending = 1;
            source_data = value;
        end
        in_valid = source_pending;
        in_data = source_data;
        out_ready = want_pop;
        #1ns;
    endtask
endmodule
```

```

    check_interface();
    take_input = in_valid && in_ready;
    take_output = out_valid && out_ready;
    if (holding && (!out_valid || out_data != held_data))
        $fatal(1, "DEPTH=%0d output changed under backpressure", DEPTH);
    holding = out_valid && !out_ready;
    held_data = out_data;
    if (in_valid && !in_ready) blocked++;
    @(posedge clk);
    if (take_output) begin head++; delivered++; end
    if (take_input) begin
        if (tail >= 8192) $fatal(1, "Reference storage exhausted");
        expected[tail] = in_data;
        tail++;
        accepted++;
        source_pending = 0;
    end
    if (take_input && take_output) simultaneous++;
    last_push = take_input;
    #1ns;
    check_interface();
endtask

task automatic reset_fifo();
    @(negedge clk);
    rst_n = 0;
    in_valid = 0;
    out_ready = 0;
    source_pending = 0;
    holding = 0;
    head = 0;
    tail = 0;
    #1ns;
    check_interface();
    repeat (2) @(negedge clk);
    rst_n = 1;
endtask

initial begin
    int unsigned random_word;
    reset_fifo();
    repeat (3) cycle(0, 1, '0); // Empty reads must not become transfers.
    for (int i = 0; i < DEPTH; i++) cycle(1, 0, W'(i + 10));
    repeat (3) cycle(1, 0, W'(99)); // Full stall, preserve pending input.
    cycle(1, 1, W'(77)); // Full FIFO reads but rejects this input edge.
    if (last_push) $fatal(1, "Full FIFO unexpectedly accepted input");
    repeat (DEPTH + 3) cycle(0, 1, '0);
    repeat (20) cycle(1, 1, W'(next_random()));
    cycle(1, 0, W'(42));
    reset_fifo(); // Discard buffered and pending transactions by contract.
    for (int i = 0; i < 600; i++) begin
        random_word = next_random();
        cycle(random_word[0], random_word[1], W'(random_word >> 8));
    end
    repeat (DEPTH + 3) cycle(0, 1, '0);
    if (head != tail || source_pending) $fatal(1, "Not drained");
    if (blocked == 0 || accepted < 30 || delivered < 30)
        $fatal(1, "Required scenarios not exercised");
    if (DEPTH > 1 && simultaneous == 0) $fatal(1, "No simultaneous transfers");
    $display("PASS FIFO W=%0d DEPTH=%0d accepted=%0d delivered=%0d both=%0d blocked=%0d",
        W, DEPTH, accepted, delivered, simultaneous, blocked);
    done = 1;
end
endmodule

module tb_fifo;
    timeunit 1ns;
    timeprecision 1ps;
    wire done1, done3, done8;
    fifo_case #(W(1), .DEPTH(1)) u1(.done(done1));

```

```

fifo_case #(.W(8), .DEPTH(3)) u3(.done(done3));
fifo_case #(.W(13), .DEPTH(8)) u8(.done(done8));
initial begin
    wait (done1 && done3 && done8);
    $display("PASS ALL FIFO CASES");
    $finish;
end
initial begin
    #100us;
    $fatal(1, "Global timeout");
end
endmodule
`default_nettype wire

```

Запуск и ожидаемый результат

Сохраните два листинга в `sync_fifo.sv` и `tb_fifo.sv`. При установленном Icarus Verilog команды для этой пары файлов:

```

iverilog -g2012 -s tb_fifo -o fifo.vvp sync_fifo.sv tb_fifo.sv
vvp fifo.vvp

```

Ожидаются три строки `PASS FIFO` для `W/DEPTH = 1/1, 8/3, 13/8` и итоговая `PASS ALL FIFO CASES`. Ошибка данных, интерфейса или зависание вызывает `$fatal` и не должна считаться успешным тестом. Стенд не требует поддержки классов, constraint solver или SVA; их самостоятельные примеры находятся в проверочной части курса.

Самостоятельное развитие

Следующая полезная модификация — разрешить одновременную запись при полном FIFO и чтении. Изменяется зависимость `in_ready`, условия записи и тест на границе. Затем можно сделать зарегистрированный выход для RAM. Каждая модификация сначала требует обновить контракт и эталонную модель, затем RTL.

Упражнение 22

При полном FIFO `in_valid=1, out_ready=1`. Какие операции примет текущая реализация на ближайшем фронте? Сформулируйте новое условие `in_ready` для варианта с замещением, не забыв про сброс.

23. Тестбенчи, clocking и ввод-вывод

Тестбенч создаёт входы, наблюдает выходы и сравнивает их с контрактом. Печать значений сама по себе не является проверкой. Самопроверяющийся тест должен завершиться успехом или ошибкой автоматически и иметь ограничение времени.

Минимальный стенд

```
module register_tb;
    timeunit 1ns;
    timeprecision 1ps;
    logic clk = 0;
    logic rst_n = 0, en = 0;
    logic [7:0] d = 0, q;
    enabled_register dut(.*);
    always #5ns clk = ~clk;
    initial begin
        repeat (2) @(negedge clk);
        rst_n = 1;
        en = 1;
        d = 8'hA5;
        @(negedge clk);
        if (q !== 8'hA5) $fatal(1, "Register mismatch");
        $display("PASS");
        $finish;
    end
    initial begin
        #1us;
        $fatal(1, "Timeout");
    end
endmodule
```

Этот стенд компилируется вместе с модулем из главы 01. Стимулы и проверка на negedge удалены от рабочего posedge DUT. !== замечает X/Z. \$fatal сообщает ошибку и завершает прогон; \$finish завершает нормальную симуляцию. \$stop часто переводит инструмент в интерактивный режим и неудобен в CI.

Driver, monitor, scoreboard

Driver преобразует транзакцию в сигналы. Monitor восстанавливает транзакции из реально принятых передач. Scoreboard сравнивает их с эталоном. Если эталон пополняется по намерению driver, а не по фактическому handshake, тест ошибочно ожидает данные, которые DUT ещё не принял.

Эталонная модель должна быть проще и независимее RTL. Для FIFO достаточно программной очереди; копирование тех же указателей и формул создаёт риск повторить один дефект в DUT и проверке.

Clocking block

```
// Фрагмент тестового интерфейса.
clocking cb @(posedge clk);
    default input #1step output #0;
    input ready, response;
    output valid, request;
endclocking
```

input здесь — сигнал, наблюдаемый тестбенчем, output — управляемый им. #1step выбирает вход непосредственно перед тактовым событием, что даёт согласованный снимок. Output skew задаёт планирование драйва относительно события. Это не физическая задержка элемента.

Тестовый процесс использует `@(cb)` и доступ через `cb.request`, а не смешивает clocking и произвольные `@(posedge clk)` на тех же сигналах. Default clocking даёт событие для разрешённых сокращений `##N` в соответствующем контексте. `##` в clocking-процедуре и в SVA имеют связанные с тактами, но различные синтаксические роли.

`program` предназначен для Reactive-исполнения тестовых процессов. Он не нужен для каждого теста: модульный тестбенч с дисциплиной clocking/событий тоже корректен. `program` не разрешает произвольные конфликтующие драйверы и не исправляет неверный момент наблюдения результата.

Файлы, аргументы и диагностика

`$fopen` открывает файл, `$fclose` закрывает, `$fdisplay`/`$fwrite` пишут, `$fgets` и `$fscanf` читают. Проверяйте handle и число успешно разобранных полей. Для памяти используйте `$readmemh`/`$readmemb`, для текстового форматирования — `$sformatf`. `%h`, `%b`, `%d`, `%s`, `%p`, `%t` подходят разным представлениям.

`$test$plusargs("TRACE")` проверяет опцию запуска, `$value$plusargs("SEED=%d", seed)` читает значение. Seed и набор параметров надо сохранять в отчёте, иначе случайный дефект трудно воспроизвести. `$dumpfile`/`$dumpvars` создают VCD там, где это поддерживается инструментом. Осциллограммы помогают разбору, но не заменяют автоматическое сравнение.

Упражнение 23

Почему тест с `if (q != expected)` способен не сообщить ошибку при X? Какие четыре сведения надо сохранить, чтобы воспроизвести случайный прогон?

24. Параллельные процессы и синхронизация

Модульные `initial/always` уже работают параллельно. `fork` позволяет создать дочерние процессы внутри процедуры. Родитель и дети разделяют доступные объекты, поэтому время жизни данных и способ передачи результата нужно определить явно.

`join, join_any, join_none`

`fork ... join` ждёт все дочерние процессы. `join_any` ждёт первый завершившийся, остальные продолжают работу. `join_none` сразу позволяет родителю двигаться дальше; порождённые процессы начинают исполнение, когда родитель приостановится или завершится.

```
// Фрагмент задачи тестбенча, изолирующая область для тайм-аута.
fork
  begin
    fork
      begin wait (done); end
      begin #1us; $fatal(1, "Operation timeout"); end
    join_any
    disable fork;
  end
join
```

Изолирующий внешний процесс ограничивает набор детей, которые попадут под `disable fork`. В произвольном большом `driver` такой оператор иначе может остановить не только таймер, но и другие дочерние работы текущего процесса. После `join_any` оставшаяся ветвь не исчезает автоматически.

`wait fork` ждёт дочерние процессы вызывающего процесса. `disable name` останавливает именованный блок/задачу по правилам языка; при повторных параллельных активациях это может иметь более широкий эффект, чем ожидается.

Захват переменных цикла

```
// Фрагмент тестбенча.
for (int i = 0; i < 4; i++) begin
  automatic int id = i;
  fork
    $display("worker %0d", id);
  join_none
end
wait fork;
```

Отдельный `automatic id` сохраняет индекс данной итерации до запуска ребёнка. Если все дети читают одну изменяемую `i` после окончания цикла, они могут увидеть конечное значение. Область объявления и момент инициализации существеннее слова `automatic`, добавленного в случайное место.

Events и ожидание

`event e`; создаёт объект события. `-> e` немедленно запускает ожидающих на событии, `->> e` использует неблокирующее планирование. `@e` ждёт будущее срабатывание и может пропустить уже случившееся в том же временном слоте. `wait(e.triggered)` использует признак срабатывания, сохраняющийся в течение слота, и помогает избежать этой конкретной гонки.

`wait(expr)` проверяет уровень условия и ждёт его истинности. `@(posedge x)` ждёт событие перехода. `wait_order(e1, e2)` проверяет порядок именованных событий с действиями успеха/ошибки. Ни один из этих операторов не создаёт автоматически очередь исторических событий: для передачи нескольких объектов нужна коллекция.

Mailbox, semaphore, process

Типизированный mailbox #(transaction_t) передаёт транзакции между процессами. put/get/peek могут блокироваться, try_put/try_get/try_peek возвращают результат попытки. Ограниченная mailbox позволяет моделировать backpressure. Если передаётся handle класса, отправитель и получатель видят один объект, пока не сделана явная копия.

semaphore управляет ключами доступа: get(N) ждёт N ключей, put(N) возвращает, try_get(N) пробует без ожидания. Это не автоматическое обеспечение честности любого пользовательского протокола; забытый возврат ключа создаёт зависание.

process::self() возвращает handle процесса. Через status, suspend, resume, kill, await можно тонко управлять выполнением. Остановка процесса не является универсальным механизмом освобождения всех приобретённых им ресурсов; завершение работы должно быть частью архитектуры теста.

Упражнение 24

После join_any завершилась рабочая ветвь, таймер остался активен. Что произойдёт через его интервал, если ничего не сделать? Почему disable fork следует изолировать?

25. Классы и объектная модель

Класс описывает программный объект тестбенча: транзакцию, драйвер, монитор, эталонную модель. Переменная класса содержит handle, первоначально null. new создаёт объект; объявление handle само по себе объекта не создаёт. Классы не входят в обычное переносимое подмножество RTL-синтеза.

Поля, методы и конструктор

```
class packet;
  rand bit [7:0] data;
  int unsigned sequence_id;

  function new(int unsigned id = 0);
    sequence_id = id;
  endfunction

  function packet copy();
    packet result = new(sequence_id);
    result.data = data;
    return result;
  endfunction
endclass
```

Поле rand будет участвовать в рандомизации, которая вводится в следующей главе. Методы класса имеют автоматическое время жизни локальных переменных. this обозначает текущий объект и помогает отличить поле от одноимённого аргумента. Конструктор new не имеет обычного возвращаемого типа.

packet a, b; a = new; b = a; создаёт один объект и два handle на него. Изменение b.data видно через a.data. Вариант b = a.copy(); выше создаёт отдельный объект. Если в классе есть handles на другие классы, такая копия должна явно решить, копировать дочерние объекты или делить их. Встроенное shallow-copy создание через new source также не выполняет произвольную глубокую копию графа объектов.

Доступ и статические члены

По умолчанию члены доступны извне. local ограничивает доступ классом, protected допускает доступ производных классов. static-поле общее для всех объектов данного класса; оно подходит счётчику выданных идентификаторов, но может случайно связать независимые тесты.

Статический метод вызывается через ClassName::method() и не имеет this. Метод можно объявить extern в классе и определить вне через квалифицированное имя, сохранив сигнатуру. const-поля имеют специальные правила установки, которые позволяют выразить неизменяемую часть объекта.

Наследование и виртуальный вызов

```
virtual class predictor;
  pure virtual function int expected(input int value);
endclass

class increment_predictor extends predictor;
  virtual function int expected(input int value);
    return value + 1;
  endfunction
endclass
```

Абстрактный virtual class нельзя непосредственно создать. pure virtual задаёт обязательный метод без реализации. Виртуальный вызов через handle базового класса выбирает реализацию фактического объекта. Без виртуальности выбор метода не обязательно соответствует динамическому типу.

`extends` поддерживает одиночное наследование реализации. `super` обращается к базовому классу, в том числе к его конструктору. Аргументы базового конструктора должны быть переданы по допустимому синтаксису. Присваивание `handle` производного класса базовому безопасно; обратное направление требует проверки `$cast`. Это `downcast handle`, а не преобразование объекта по полям.

Обобщения и интерфейсные классы

Класс может иметь параметры значений и типов: `class box #(type T = int);`. Разные специализации образуют разные типы. `Static`-члены относятся к конкретной специализации, что важно для общих счётчиков и фабрик.

`interface class` задаёт набор чисто виртуальных методов; класс может `implements` несколько таких контрактов. Это не аппаратный `interface`, не набор сигналов и не способ множественного наследования полей реализации. `Forward typedef` позволяет объявить имя класса до полного определения, например для взаимных ссылок.

В 2023 добавлены средства ограничения наследования/переопределения и `std::weak_reference`. Слабая ссылка позволяет наблюдать объект без той же роли удержания, что обычный `handle`. Эти возможности требуют режима 2023; они не нужны для простого `driver` и описаны в карте обновлений главы 33.

Связь с сигналами

Класс драйвера хранит `virtual stream_if` или другой `virtual-interface handle`. Его передают из модульного окружения после создания физического интерфейса. Так класс оперирует транзакциями, а доступ к сигналам централизован в драйвере. Не распределяйте запись одного сигнала между несколькими независимыми объектами.

Упражнение 25

Драйвер кладёт `handle` пакета в `mailbox` и затем изменяет его для следующей передачи. Почему монитор/`scoreboard` может увидеть новые данные в старой транзакции? Какие два подхода устраняют проблему?

26. Случайные воздействия и ограничения

Случайный тест полезен, если генерирует допустимые транзакции, воспроизводится и проверяет результат. Равномерное создание битов без учёта протокола часто тратит почти всё время на бессмысленные или запрещённые комбинации.

rand, randc и constraint

```
class request;
  rand bit write;
  rand bit [7:0] address;
  rand int unsigned length;
  constraint legal {
    address inside {[8'h20:8'h7f]};
    length inside {[1:8]};
    address + length <= 128;
    if (write) length <= 4;
  }
endclass
```

Это декларация отношений: solver ищет значения, удовлетворяющие всем активным ограничениям одновременно. Порядок строк не является порядком присваиваний. Здесь широкий length делает арифметику суммы достаточной; при узких полях их нужно явно расширить, чтобы переполнение не изменило смысл ограничения.

rand выбирает новое значение с учётом ограничений. randc циклически перебирает значения своего допустимого пространства до повторения, с ограничениями взаимодействия solver и размера типа. Это не гарантия обхода всех комбинаций нескольких полей и не средство полного покрытия теста.

Вызов и обработка отказа

```
// Фрагмент initial после объявления класса request.
request req;
req = new;
if (!req.randomize() with { length == 2; })
  $fatal(1, "Unsatisfiable request constraints");
```

Неудача randomize() должна быть обработана: использовать прежние значения как новые случайные данные — скрытая ошибка теста. Не помещайте вызов с нужным побочным эффектом только внутрь отключаемого immediate assert. При выключенных проверках такой flow может перестать выполнять нужную рандомизацию.

std::randomize(x, y) with {...} работает с переменными вне класса. \$urandom, \$urandom_range дают процедурную случайность без solver. У \$urandom_range первым аргументом обычно выступает максимум, вторым минимум; проверяйте порядок, а не переносите привычку из другой библиотеки.

Распределения и порядок решения

dist задаёт веса значений и диапазонов. := назначает указанный вес каждому значению диапазона, :/ распределяет общий вес диапазона между значениями. Это существенно, если диапазоны разной длины.

solve a before b влияет на распределение решений, а не превращает ограничение в процедурную программу и не разрешает несовместимые условия. soft задаёт предпочтение, которое может быть отброшено при более сильном конфликтующем требовании; оно удобно для значений по умолчанию в базовом тесте. disable soft управляет такими предпочтениями в разрешённых контекстах.

`rand_mode` включает/выключает участие поля; `constraint_mode` — блока ограничений. Отключённое случайное поле становится фиксированным входом для оставшихся активных ограничений. `pre_randomize` и `post_randomize` позволяют подготовить состояние и вычислить производные данные; не прячьте там изменение контракта.

Массивы и последовательности

Можно ограничить `array.size()`, затем задать поэлементные условия через `foreach`. Редукции `sum()` `with (...)` полезны для общей длины/суммы, но требуют правильной ширины. `unique { ... }` в `constraint` задаёт различие значений; это другой контекст, чем `unique case`.

`randcase` случайно выбирает ветвь по весу. `randsequence` описывает грамматику случайной последовательности продуктов. Они полезны для сценариев, но не заменяют `solver` ограничений над всеми полями транзакции.

`Seed` обеспечивает воспроизводимость только при согласованном коде, инструменте, порядке создания объектов и потоков случайности. Добавление лишнего вызова генератора может изменить последующую трассу. Сохраняйте `seed`, версию симулятора, параметры теста и ревизию исходников.

Упражнение 26

У адреса допустимы 0..15, длина 1..4, транзакция не должна пересекать границу 16. Запишите ограничения с достаточной шириной суммы. Почему один `seed` без версии исходников не гарантирует воспроизведение дефекта?

27. Assertions, sequences и properties

Assertion делает требование исполняемым. Immediate assert проверяет условие при выполнении процедуры. Concurrent assertion наблюдает последовательность значений на событиях и подходит временным свойствам протокола. В симуляции нарушение сообщает об ошибке; в formal тот же контракт может служить целью доказательства или ограничением окружения.

Immediate и deferred проверки

```
// Фрагмент тестбенча.  
assert (received == expected)  
    else $error("Unexpected data");
```

Ноль, X или Z в условии означают неуспех. Простая immediate-проверка видит значения текущего выполнения и может наблюдать промежуточное состояние комбинационной сети. Deferred-формы assert #0 и assert final откладывают оценку/сообщение по специальным правилам областей планировщика. Это не то же, что вручную поставить #0 перед любой проверкой.

assert проверяет, assume задаёт допущение (особенно в formal), cover наблюдает достижение. Использование assume для свойства DUT может сделать «доказательство» бессодержательным: вы предположили именно то, что хотели доказать.

Базовая временная запись

```
// Фрагмент модуля, сигналы объявлены в его интерфейсе.  
property response_in_time;  
    @(posedge clk) disable iff (!rst_n)  
        request |-> ##[1:3] response;  
endproperty  
assert property (response_in_time);  
cover property (@(posedge clk) rst_n && request ##2 response);
```

Для каждого такта с request начинается отдельная попытка. После него response должен встретиться через 1, 2 или 3 такта. |-> начинает consequent с такта окончания antecedent; |=> сдвигает старт consequent на следующий такт. Поэтому для однотокового antecedent a |=> b соответствует a |-> ##1 b.

Конкурентные проверки обычно выбирают значения в Preponed, а оцениваются позже в Observed. Они не видят только что записанное NBA-состояние как «значение на том же фронте». disable iff является условием асинхронного прерывания попыток и не тождественно sampled условию antecedent.

Sequences и повторения

Sequence описывает совпадение во времени, property — требование, например импликацию между совпадениями. Их можно именовать, параметризовать и повторно использовать. Временные интервалы задаются в тактах clocking event.

Конструкция	Идея
a ##N b, ##[m:n]	Задержка между совпадениями
a[*N], a[*m:n]	Последовательное повторение
a[->N]	Goto-повторение до N-го истинного события
a[=N]	Непоследовательное повторение с иной допустимой точкой окончания
s1 and s2, s1 or s2	Совместные/альтернативные последовательности

Конструкция	Идея
<code>s1 intersect s2</code>	Совпадают начало и окончание
<code>expr throughout s</code>	Условие сохраняется в течение последовательности
<code>s1 within s2</code>	Вложенное совпадение внутри другого
<code>first_match(s)</code>	Выбор первого подходящего окончания

Различия `[ -> ]` и `[ = ]` особенно важны при добавлении последующего `##1`: точка окончания определяет, относительно чего начинается следующий фрагмент. Пустые совпадения, допускаемые повторением ноль раз, нужно анализировать отдельно.

Sampled-value функции

`$past(x, N)` читает прежнее выбранное значение, `$stable(x)` проверяет неизменность, `$changed(x)` — изменение. `$rose/$fell` смотрят переход младшего бита аргумента; для многобитного слова это не «любое увеличение». `$sampled(x)` полезна в сообщении об ошибке, чтобы вывести именно значение проверки, а не уже изменившийся сигнал Reactive-области.

Полезное свойство `valid/ready`:

```
assert property (@(posedge clk) disable iff (!rst_n)
    out_valid && !out_ready | => out_valid && $stable(out_data));
```

Оно требует сохранения элемента в следующий такт после остановки. Для первого использования `$past` нужна понятная начальная история: дополнительные `valid`-флаги истории или явно оговорённое поведение после `reset`.

Сильные, слабые и составные свойства

Если `antecedent` не возникает, `implication` может пройти вакуозно. Добавьте `cover` на запуск сценария и на его успешное завершение. `strong` требует завершённого совпадения там, где слабое ожидание могло остаться открытым к концу теста. Бесконечный диапазон `##[1:$]` без анализа силы свойства способен скрыть отсутствие ответа в конечной симуляции.

Семейства `until`, `s_until`, `until_with`, `s_until_with`, `nexttime`, `s_nexttime`, `always`, `s_always`, `eventually`, `s_eventually` описывают длительность и обязательность событий. `accept_on/reject_on` и синхронные варианты управляют прерыванием; `iff`, `implies`, `and`, `or`, `not`, условные свойства комбинируют требования. Их грамматику нельзя механически заменять обычной булевой алгеброй над сигналами.

Есть локальные переменные `sequence`, многотактовые/многодоменные свойства, `global clocking` и `global sampled-value` функции. Их применение требует явно заданных событий и поддержки `formal/simulator`. Для повторного использования служат `checker`, `default clocking`, `default disable iff` и `bind`. Поддержку SVA проверяйте по конкретному инструменту; принятие синтаксиса не доказывает, что все проверки включены в запуске.

Упражнение 27

Чем отличаются `req | -> ack` и `req | => ack`? Почему тест без единого `req` не подтверждает обработку запроса даже при отсутствии нарушений `assertion`?

28. Функциональное покрытие

Проверка отвечает «верно ли поведение», покрытие — «какие предусмотренные ситуации мы наблюдали». 100% покрытия не доказывает корректность: план мог не включить существенный сценарий, а наблюдаемое событие могло быть ошибочным.

Covergroup и coverpoint

```
class fifo_coverage;
  covergroup occupancy with function sample(int used, bit push, bit pop);
    option.per_instance = 1;
    fill: coverpoint used {
      bins empty = {0};
      bins middle = {[1:7]};
      bins full = {8};
      illegal_bins invalid = default;
    }
    put: coverpoint push;
    get: coverpoint pop;
    actions: cross put, get;
  endgroup

  function new();
    occupancy = new;
  endfunction
endclass
```

Пример относится к FIFO глубины 8. Sample вызывают один раз на наблюдаемую операцию с согласованным снимком count и фактических handshake. Для другого DEPTH нужно изменить или параметризовать bins; иначе «full» потеряет смысл.

Covergroup можно выбирать по событию в заголовке или явно через sample(). Явный вызов удобен для транзакций: monitor сначала распознаёт факт принятия, потом регистрирует его параметры.

Bins и переходы

Bin — счётчик попаданий в заданное множество. bins low = {[0:3]}; создаёт один bin для диапазона; bins each[] = {[0:3]}; — набор отдельных bins. Автоматические bins удобны для небольшого пространства, но могут маскировать недостаточно продуманный план.

Transition bins вроде (0 => 1 => 0) наблюдают последовательные sampled значения. Можно описывать повторения и wildcard-шаблоны. Это средства измерения сценария, а не полная замена временным assertions с их правилами clocking.

ignore_bins исключает значения из цели покрытия. illegal_bins отмечает недопустимые значения, но не является универсальной заменой assertions: контроль ошибок и политика инструментов различаются. iff разрешает выборку в заданном состоянии; он не удаляет автоматически все невозможные комбинации из определения cross.

Cross coverage

Cross измеряет сочетания bins нескольких coverpoints. Для двух признаков push/pop важны четыре сочетания, особенно одновременная передача. Cross адреса, длины, режима, состояния и типа команды может быстро создать огромное число комбинаций, многие из которых физически невозможны.

Отбирайте значимые сочетания через binsof, intersect, with в допустимых контекстах, исключайте архитектурно невозможные варианты с объяснением. Нельзя исключать bin только потому, что генератор пока не научился его достигать. В 2023 появились дополнительные средства управления сохранением

автоматически созданных cross bins; их поддержку надо сверять отдельно.

Опции, экземпляры и отчёт

`option` настраивает конкретный экземпляр, `type_option` — уровень типа `covergroup`. Среди задач настройки: `per-instance` учёт, `weight`, `goal`, `at_least`, имена и комментарии. Порог попаданий определяет, сколько наблюдений достаточно для bin, но повторение одной ситуации тысячу раз не расширяет набор сценариев.

Методы `get_coverage`, `get_inst_coverage`, `start`, `stop` помогают управлять сбором. Слияние баз между прогонами зависит от инструмента и совместимости модели покрытия. Сохранённый `seed` объясняет трассу, а сохранённая `coverage`-база показывает вклад прогона в общую цель.

План покрытия для FIFO

Измеряйте пустоту, заполненность, переход к `full`, переход к `empty`, `wrap` обоих указателей, одновременные передачи, несколько тактов `backpressure`, сброс непустого буфера и минимальные параметры.

Функциональное покрытие дополняйте `code coverage: statements, branches, toggles, FSM` — набор зависит от инструмента.

Невыполненная ветвь кода может означать пробел теста или недостижимую логику. Невыполненный функциональный bin — пробел стимула, ошибочную модель покрытия или нарушенный контракт. Каждый остаток требует классификации по фактам.

Упражнение 28

Почему покрытие всех значений `count` не доказывает, что FIFO сохраняет порядок? Какие bins различают «буфер достиг `full`» и «источник пытался записать при `full`»?

29. DPI, VPI и границы UVM

SystemVerilog может вызывать внешний код и предоставлять доступ к модели инструментальным приложениям. Это полезно для эталонных алгоритмов, файловых форматов и связи с окружением, но требует соблюдения ABI и правил симулятора.

DPI-C

```
// Объявление в модуле или пакете.  
import "DPI-C" function int add_reference(input int a, input int b);
```

Соответствующая C-функция для этих двухсостоянийных целых:

```
int add_reference(int a, int b) {  
    return a + b;  
}
```

Фрагменты показывают границу языка; чтобы выполнить вызов, C нужно собрать в библиотеку способом выбранного симулятора и загрузить в прогон. Дополнительно ограничьте диапазон входов: переполнение `signed int` в C нельзя бездумно приравнять к модульной битовой арифметике SV.

`import` объявляет внешний вызов, `export "DPI-C"` делает SV-подпрограмму доступной внешнему коду. Импортированная функция не должна потреблять модельное время. DPI-задачи применяются для взаимодействий, допускающих соответствующую временную семантику через SV, по правилам DPI и реализации.

`pure` обещает отсутствие побочных эффектов и зависимости от скрытого состояния в предусмотренном контракте. `context` нужен для взаимодействия, которому важен контекст симулятора и обратные вызовы SV. Не назначайте эти квалификаторы ради скорости, если функция не удовлетворяет их требованиям.

Типы и память на границе

Для сложных интерфейсов используйте сгенерированный симулятором DPI-заголовок и `svdpi.h`. Packed bit-векторы, четырёхсостоянийные logic-векторы, строки, open arrays и handles имеют специальные представления. `svLogicVecVal` передаёт две составляющие битового кодирования, необходимые для X/Z; это не просто C `uint32_t`.

Open arrays доступны через `svOpenArrayHandle` и функции запроса границ/элементов. Не предполагайте произвольную непрерывность, порядок индексов и layout памяти. `chandle` переносит непрозрачный указатель, но не управляет автоматически его освобождением. `string` на границе требует ясного владения и времени жизни.

Контекст `svScope` позволяет выбирать нужную SV-область в предусмотренных операциях. Нельзя безопасно вызывать API симулятора из произвольного потока ОС, если это не поддерживает документация конкретного инструмента.

VPI и интерфейс assertions

VPI предоставляет обход объектов модели, чтение/изменение значений, регистрацию callbacks и взаимодействие с планировщиком через стандартный API. Это подход для трассировщиков, адаптеров и отладочных инструментов, которым нужно обнаруживать объекты динамически. DPI обычно проще для явного вызова известной функции с заданной сигнатурой.

C VPI важны тип `handle`, область `callback` и разрешённые операции в этой области. Запись в `read-only callback` или чтение до установления NBA-значений нарушит контракт наблюдения. Полный справочник

`vpri_*`, кодов объектов и `callbacks` нужен из LRM и заголовков симулятора; его нельзя заменить краткой таблицей. Интерфейсы управления `assertions` предоставляют дополнительные средства инструментального наблюдения и контроля проверок.

UVM

UVM — библиотека классов и методология, стандартизованная отдельно как IEEE 1800.2. Она использует классы, `randomization`, `interfaces` и другие средства SystemVerilog, но имена `uvm_component`, `uvm_sequence`, `uvm_config_db` не являются ключевыми словами языка. Нужны библиотека, её версия и настройки сборки.

UVM организует `driver/monitor/agent`, `sequencer` и `sequences`, TLM-связи, `scoreboard`, фазы и сообщения. `Factory` позволяет заменять реализации, `configuration` передаёт настройки, `objections` управляют завершением фаз. Для маленького регистра прямой тестбенч проще; для семейства сложных компонентов единая методология помогает повторному использованию. API и версии берите из [официальной страницы Accellera UVM](#).

Упражнение 29

Что выбрать для вызова фиксированной C-модели CRC — DPI или обход через VPI? Почему нельзя передавать 4-state packed-вектор как обычный C-массив `bool`?

30. Примитивы, силы и пользовательские сети

RTL обычно описывает логику операторами и процессами. Для чтения netlist, моделей ячеек и общих шин нужны нижние уровни языка: примитивы, сетевое разрешение, силы драйверов и пользовательские сетевые типы.

Встроенные вентили

```
module gate_example(input wire a, b, enable, output tri y);
    wire combined;
    and u_and(combined, a, b);
    bufif1 u_driver(y, combined, enable);
endmodule
```

Для обычных логических вентилей первым терминалом идёт выход, затем входы. and, nand, or, nor, xor, xnor поддерживают логические функции; buf и not — буферизацию/инверсию. У buffer-примитивов с несколькими выходами порядок иной по числу терминалов: вход последний. Не переносите правило произвольного модуля на каждый вид примитива.

bufif1/bufif0, notif1/notif0 имеют управление включением. Когда драйвер отключён, выход Z. Массив примитивов создаёт повторяющиеся элементы для векторов. Имя экземпляра и диапазон помогают навигации по netlist.

Сети и разрешение

wire и tri используют обычное разрешение драйверов. wand/triand разрешают по проводному AND, wor/trior — по проводному OR. tri0/tri1 дают слабое значение по умолчанию, supply0/supply1 моделируют постоянные питающие уровни. trireg моделирует хранение заряда с размером/затуханием в соответствующих режимах; это не обычный регистр reg.

Силы вроде supply, strong, pull, weak участвуют в разрешении конфликтующих драйверов. Более сильный может определить результат, равносильные противоположные значения дают X. Сила не равна непрерывному напряжению: это дискретная модель. pullup/pulldown создают подтягивающие драйверы.

uwire выражает требование одного драйвера. interconnect применяется для обобщённых межсоединений по специальным правилам вывода типа и соединения; его не следует использовать как замену любой переменной. alias делает сетевые части одной связью и поддерживает двунаправленность, в отличие от однонаправленной зависимости assign.

Переключательный уровень

nmos, pmos, cmos и резистивные варианты моделируют управляемые переключатели. tran, rtran, tranif0/1, rtranif0/1 описывают двунаправленное соединение. Правила передачи силы, X и Z зависят от примитива. Такая модель полезна для специализированных библиотек, но не является SPICE и не описывает аналоговый ток транзистора.

Задержки вентилей могут различать нарастание, спад и выключение. Выбор мин/тип/макс и взаимодействие путей рассматриваются в следующей главе. Для обычной FPGA-логики используйте RTL или рекомендованные vendor primitives; транзисторные конструкции часто не поддерживаются синтезом.

UDP: пользовательские примитивы

```
primitive udp_and(output y, input a, b);
  table
    0 ? : 0;
    ? 0 : 0;
    1 1 : 1;
  endtable
endprimitive
```

UDP имеет скалярный выход и скалярные входы; таблица задаёт комбинационное или последовательностное поведение. Неописанные комбинации дают X; ? в таблице покрывает допустимые входные состояния, а Z обрабатывается по UDP-правилам, обычно как неизвестность на входе.

У последовательностного UDP выход объявляют хранящим состояние. Строка таблицы содержит входы, текущее состояние и следующее состояние. - означает сохранение состояния, обозначения переходов вроде (01), r, f, p, n, * описывают события на входах с разной точностью. Можно сочетать уровневые условия с переходами, соблюдая ограничения таблицы. Начальное значение задаётся допустимым способом для последовательностного UDP.

UDP — компактная таблица цифровой ячейки, но не произвольная процедура: нет широких шин, задач и вычислительных циклов внутри таблицы. Экземпляры создают структурно, как примитив, с правильным порядком терминалов.

nettype

nettype позволяет связать тип данных сети с пользовательской функцией разрешения. Она получает набор значений драйверов и вычисляет результат. Это полезно для абстрактных моделей электрических соединений или смешанных доменов данных. Функция должна соблюдать ограничения языка для разрешения; побочные временные действия здесь неуместны.

Пользовательский nettype с real не превращает SystemVerilog в полный аналоговый решатель. Для аналогового моделирования существуют отдельные стандарты и инструменты. Поддержку UDN и синтеза проверяют независимо.

Упражнение 30

Два драйвера wire равной силы выдают 0 и 1. Какой результат? Что будет, если один драйвер выдаёт Z? Чем сетевое разрешение отличается от двух записей в одну переменную из разных процессов?

31. specify, задержки и библиотечные модели

RTL отвечает на вопрос о логической функции и последовательности тактов. Модель ячейки дополнительно описывает время распространения и ограничения входных событий. Эти средства важны для gate-level симуляции и чтения библиотек.

Виды задержек

Задержка может принадлежать непрерывному присваиванию, примитиву, сети, процедурному оператору или пути в `specify`. У этих конструкций разная семантика планирования и фильтрации импульсов. Нельзя считать любое `#N` идеальным транспортным проводом с одинаковым поведением.

Вентильные и непрерывные задержки обычно подавляют короткие импульсы по инерционным правилам соответствующей конструкции. Неблокирующее присваивание с внутренней задержкой планирует отдельное обновление выбранного значения; оно часто используется для иной модели передачи событий. Перед моделированием узких импульсов сверяйте точные правила LRM.

Запись `min:typ:max` предоставляет три значения задержки; какое из них используется, выбирается настройками симуляции. Это не три параллельных моделирования статистического разброса.

Specify paths

```
module timed_and(input wire a, b, output wire y);
    timeunit 1ns;
    timeprecision 1ps;
    and (y, a, b);
    specify
        specparam T_RISE = 0.2, T_FALL = 0.3;
        (a => y) = (T_RISE, T_FALL);
        (b => y) = (T_RISE, T_FALL);
    endspecify
endmodule
```

`specparam` задаёт параметры `timing`-модели. Путь `=>` и `full-connection` путь `*>` имеют разные правила сопоставления портов. Пути могут зависеть от состояния через `if/ifnone` и учитывать определённые переходы. Это отдельная грамматика, а не произвольный процедурный `if`.

У синтезируемой логической функции `and` наличие `specify` не меняет булеву таблицу. Синтез обычно не выводит из этих чисел физически гарантированную задержку: реальные значения берутся из технологических библиотек и `timing-flow`. Некоторые симуляторы требуют явного включения `specify`-обработки.

Timing checks

Семейства `$setup`, `$hold`, `$setuphold` проверяют взаимное расположение событий данных и такта. `$recovery`, `$removal`, `$recrem` относятся, например, к асинхронному сбросу. `$width`, `$period`, `$skew`, `$timeskew`, `$fullskew`, `$nochange` охватывают ширины импульсов, период и согласование событий.

Аргументы содержат события, ограничения, условия и иногда `notifier`. `Notifier` может измениться при нарушении, а модель ячейки — распространить `X`, показывая недостоверность состояния. Это реакция модели на нарушение, а не физическая форма метастабильности во всех деталях.

Для импульсных путей существуют средства `pulstyle_oneevent`, `pulstyle_ondetect`, `showcancelled`, `noshowcancelled` и `pulse limits`. Они управляют отражением отменённых/коротких импульсов; применять их следует вместе с документацией конкретной библиотечной модели.

SDF и back-annotation

SDF хранит задержки и timing-ограничения вне HDL. `$sdf_annotate` и настройки симулятора сопоставляют их экземплярам netlist. Имена иерархии, пути, версия библиотеки и единицы времени должны совпасть. Сообщение о частично неприменённых данных аннотации означает неполную timing-модель.

Gate-level simulation помогает обнаружить проблемы запуска, X-поведения, аннотации и временных сценариев. Статический временной анализ исследует другие свойства и не заменяется одним динамическим прогоном. Для sign-off нужен согласованный flow, а не самостоятельная задержка в учебном RTL.

Упражнение 31

Почему добавление #2 к выходу RTL не гарантирует двухнаносекундную задержку после place-and-route? Что нужно проверить, если SDF сообщает о ненайденном экземпляре?

32. Библиотеки и конфигурации

Разные библиотеки могут содержать модули с одним именем: поведенческую модель, RTL и технологическую реализацию. Конфигурация позволяет выбрать, какие определения образуют конкретную элаборированную иерархию.

Library map и config

Схематический library map, пути должны соответствовать реальному проекту:

```
library work ./rtl/*.sv;  
library fast_cells ./models/fast/*.sv;
```

Конфигурация с выбором реализации одного экземпляра:

```
config simulation_config;  
  design work.top;  
  default liblist work;  
  instance top.u_memory use fast_cells.memory_model;  
endconfig
```

design определяет вершину, default liblist — список поиска по умолчанию, instance задаёт правило для конкретного пути. cell применяется к типу ячейки в предусмотренной области. use выбирает реализацию явно, liblist задаёт поиск. Выбор конфигурации в командной строке зависит от инструмента.

Это учебная иллюстрация: она не компилируется как самостоятельная схема без модулей top, memory_model и реального library map. Не нужно создавать фиктивную память, чтобы скрыть эти зависимости.

Иерархические конфигурации

Конфигурация может ссылаться на другую конфигурацию с соответствующим :config-обозначением. Так интегратор выбирает заранее определённую сборку подсистемы. Правила библиотечного поиска и приоритет правил instance/cell нужно читать вместе: локальное правило может изменить ожидаемый выбор реализации.

Список библиотек не является препроцессорным include path. Первый ищет определения элементов проекта, второй — текст включаемых файлов. Путать их особенно легко при переходе между GUI-проектом и файловой сборкой.

Параметры и интерфейсы

Конфигурационные правила языка допускают определённые формы выбора параметров при binding, но практическая поддержка неодинакова. Для основного учебного RTL предпочтительна явная параметризация экземпляра #(.W(. . .)): она локальна и хорошо читается. Конфигурация полезна для выбора реализаций, а не для скрытой подмены размера протокола в неожиданном месте.

Замещающий модуль обязан соответствовать ожидаемому интерфейсу и временному контракту. Совпадение имён портов не доказывает эквивалентность задержки, сброса или режима read-during-write.

extern module объявляет интерфейс внешнего определения; nested-модули ограничивают область определения. Их роль отличается от config: прототип не выбирает библиотеку, а config не описывает поведение модуля.

Воспроизводимая сборка

Храните файловый список, порядок пакетов, `include directories`, `defines`, `library map`, верхний модуль, параметры и версию инструмента. Именно весь этот набор определяет элаборацию. Фраза «скомпилировать все `.sv`» недостаточна, если в дереве есть альтернативные реализации или несколько тестовых вершин.

Для небольшого проекта файловый список и разные `top`-модули часто проще конфигураций. Когда используются `config`, добавьте в лог распечатку связанной иерархии и проверьте выбор ячеек. Поддержка конфигураций компилятором не означает их одинаковую реализацию всеми средствами синтеза.

Упражнение 32

Один экземпляр памяти должен использовать быструю модель, остальные — RTL. Какое правило точнее: `instance` или общее `cell`? Какие параметры запуска необходимо хранить вместе с исходниками для воспроизводимости?

33. Стандарты, изменения 2023 и рабочий процесс

Редакция языка, режим компилятора и синтезируемое подмножество — три разные характеристики проекта. Для новых конструкций нужна явная проверка версии, а для аппаратного проекта ещё и подтверждение синтеза нужной структуры.

Нормативная база после 2022 года

По карточке IEEE, **IEEE 1800-2023** активен, утверждён 6 декабря 2023 года и опубликован 28 февраля 2024 года. Он заменяет 1800-2017; PDF доступен через IEEE GET при поддержке Accellera. Проверка источника выполнена 23 сентября 2026 года. Более позднюю редакцию эта методичка не предполагает. [Карточка стандарта](#).

Обновление содержит дополнения, исправления и уточнения. Не следует приписывать 2023 году давно существовавшие `always_ff`, интерфейсы, классы или `assertions`. Они используются в основном тексте как устоявшиеся средства языка.

Карта изменений

Для выбора тем сверены первичные `release notes` реализации `slang v6.0` и `v7.0`. Это сведения реализации об обновлении языка; при споре нормативным источником остаётся IEEE 1800-2023, а не принятый одним инструментом текст.

Область	Что появилось/расширено	Практический смысл
Строки	Многострочные литералы с тройными кавычками	Удобнее длинный текст моделей
Препроцессор	Логические условия в условных директивах	Меньше вложенных условий
Типы	Soft packed unions, ограничения типовых параметров	Более гибкие представления
Подпрограммы	<code>ref static</code>	Явное требование <code>lifetime</code> аргумента
Массивы	<code>map</code> , расширения итераторов	Преобразование коллекций
Классы	Ограничения наследования/переопределения, <code>weak references</code>	Более строгие объектные контракты
Coverage	Средства управления автоматически остающимися <code>cross bins</code>	Точнее цель покрытия

Примеры новых конструкций в [sv2023.sv](#) проверяются отдельно в режиме 1800-2023. Остальные примеры ориентированы на обычный синтаксис 2017, чтобы основной курс не зависел от новых расширений. Это выбранная политика переносимости, а не обещание поддержки любым EDA.

Пример новых типов и строк

```
// Объявления для режима SystemVerilog-2023.
typedef union soft packed {
    logic [31:0] wide;
    logic [7:0] narrow;
} overlay_t;
string help_text = ""First line
Second line"";
```

Размер объединения определяется наибольшим членом. Разноразмерные представления перекрываются по правилам soft union; операции через узкий член требуют внимания к старшим битам. Не используйте это как неявный протокол упаковки без теста ожидаемого расположения данных. Строка нужна тестбенчу, а не аппаратной памяти фиксированного формата.

Инструменты и уровни проверки

Slang выполняет разбор, семантическую проверку и элаборацию, но не моделирует время выполнения. В репозитории используется pyslang, Python-интерфейс этого компилятора. Это позволяет проверить пакеты, классы, SVA и примеры новой редакции без утверждения, что они были симулированы. [Документация slang](#).

Icarus Verilog удобен для небольших переносимых примеров; набор SV-возможностей нужно проверять по выбранной версии. Verilator поддерживает свой набор языковых возможностей и режимов моделирования; детали X/Z, coverage, assertions и timing необходимо читать для конкретной версии. [Документация Verilator](#).

Yosys и коммерческие FPGA/ASIC-синтезаторы имеют разные frontends и ограничения. Вывод RAM, интерфейсов и начальных значений проверяйте по документации производителя и отчёту синтеза. Смена расширения файла с .v на .sv не подтверждает полную поддержку 1800-2023.

Практический цикл

1. Записать интерфейс, допустимые параметры, события принятия и задержку.
2. Выполнить lint/компиляцию и элаборацию граничных конфигураций.
3. Запустить направленные тесты, затем случайные с сохранёнными seed.
4. Проверить assertions и покрытие, разобрать все предупреждения по смыслу.
5. Выполнить синтез, посмотреть ресурсы/память/защёлки и timing/CDC/RDC.
6. Сохранить исходники, команды, версии и результаты в воспроизводимом flow.

Для этой методички [VALIDATION.md](#) фиксирует только действительно выполненные проверки. Текстовые фрагменты объясняют локальную конструкцию; самостоятельные файлы examples служат базой исполняемых проверок.

Упражнение 33

Компилятор принимает class и covergroup, а синтезатор их отвергает. Есть ли противоречие стандарту языка? Какие два отдельных подтверждения нужны, чтобы заявить работоспособность нового RTL на выбранной FPGA?

A. Матрица синтезируемости

Синтезируемость определяется сочетанием конструкции, способа использования, версии инструмента и целевой технологии. Таблица — ориентир для архитектурного выбора, а не замена перечню поддержки выбранного EDA.

Конструкция	Обычное применение	Ограничения
module, статическая иерархия	RTL	Порты/размеры должны разрешаться при элаборации
parameter, localparam, generate	RTL	Константные условия, допустимые параметры
logic, wire, packed-векторы	RTL	Правила драйверов, конечная ширина
bit, целые типы	RTL/тест	Не скрывать X двухсостоянийной моделью; контролировать ширину
enum, typedef, struct packed	RTL	Кодирование/типы должны поддерживаться frontend
union packed	Представления битов	Одинаковый размер обычных членов; проверить поддержку
Распакованные фиксированные массивы	RAM/регистры	Способ чтения/записи определяет ресурсы
assign, always_comb	Комбинационная логика	Полные присваивания, отсутствие петель
always_ff	Регистры	Допустимый шаблон clock/reset, один владелец
always_latch	Намеренная защёлка	Нужна поддержка и соответствующий timing-flow
if, case, unique, priority	Выбор логики	Обещания квалификаторов должны выполняться
Процедурный for, foreach	Повторяющаяся логика	Синтезатор должен ограничить и развернуть работу
Чистые функции	Комбинационное вычисление	Без времени и неподдерживаемой динамики
Задача без времени	Возможна как RTL-абстракция	Аргументы и тело должны поддерживаться
interface, modport, пакеты	Организация RTL	Набор возможностей зависит от синтезатора
initial, начальные значения RAM	Часто FPGA-flow	Не переносимая гарантия ASIC power-up
#delay, файловые операции	Модели/тест	Не обычные аппаратные задержки и периферия
Динамические массивы, очереди, ассоциативные массивы	Тестбенч	Не обычное динамическое выделение аппаратуры
string, real, shortreal	Тест/моделирование	Специальные flow возможны, обычный RTL иной
Классы, mailbox, semaphore	Проверка	Программные объекты симуляции

Конструкция	Обычное применение	Ограничения
randomize, constraints, covergroups	Проверка	Не аппаратный генератор/измеритель по умолчанию
SVA, checker, bind	Симуляция/formal	Некоторые flow строят аппаратные мониторы отдельно
DPI/VPI	Связь с внешним кодом	Не синтезируемый вызов C-процессора
specify, SDF, timing checks	Gate-level модели	Не способ задать физический delay RTL
Силы, MOS/tran, trireg	Модели сетей	Ограниченная поддержка логическим синтезом
force/release	Отладка тестбенча	Переопределение модели, не стандартная RTL-схема
config, library map	Выбор реализации	Поддержка flow и инструментов неодинакова

Практический критерий

Для любой новой конструкции задайте четыре вопроса: сколько аппаратных элементов она требует; когда они обновляются; каковы их входы и выходы; может ли инструмент вывести этот ресурс для выбранной цели? Если ответ «во время работы создаётся объект произвольного размера», это обычно программная модель, а не переносимое RTL.

Синтезируемый текст может описывать чрезмерно дорогую схему. Например, переменное деление или большой мультиплексор разрешены далеко не бесплатно. Смотрите отчёт ресурсов и критический путь. Удалённый синтезатором сигнал может означать оптимизацию константы, недостижимую логику или ошибку соединения.

Сверка конкретной реализации: [AMD UG901](#). Инструментальное правило не следует распространять на все FPGA/ASIC.

В. Справочник ключевых слов и системных средств

Ключевые слова чувствительны к регистру и пишутся строчными буквами. Ни `$display`, ни `randomize`, ни `uvm_component` не являются зарезервированными ключевыми словами только потому, что часто встречаются в SV-коде. Режим ``begin_keywords` может менять набор распознаваемых слов.

Алфавитный указатель ключевых слов

Ниже объединённый словарь языка, включая прежние средства Verilog и специализированные временные конструкции. Смысл слова определяется контекстом: `and` бывает вентилем и оператором последовательностей; `static` — `lifetime` и член класса; `soft` — ограничение или разновидность объединения.

`accept_on alias always always_comb always_ff always_latch and assert assign
assume automatic before begin bind bins binsof bit break buf bufif0 bufif1
byte case casex casez cell chandle checker class clocking cmos config const
constraint context continue cover covergroup coverpoint cross deassign
default defparam design disable dist do edge else end endcase endchecker
endclass endclocking endconfig endfunction endgenerate endgroup endinterface
endmodule endpackage endprimitive endprogram endproperty endsequence
endspecify endtable endtask enum event eventually expect export extends
extern final first_match for force foreach forever fork forkjoin function
generate genvar global highz0 highz1 if iff ifnone ignore_bins illegal_bins
implements implies import incdir include initial inout input inside instance int
integer interconnect interface intersect join join_any join_none large let
liblist library local localparam logic longint macromodule matches medium
modport module nand negedge nettype new nexttime nmos nor noshowcancelled not notif0
notif1 null or output package packed parameter pmos posedge primitive
priority program property protected pull0 pull1 pulldown pullup
pulsestyle_ondetect pulsestyle_onevent pure rand
randc randcase randsequence rcmos real realtime ref reg reject_on release
repeat restrict return rnmos rpmos rtran rtranif0 rtranif1 s_always
s_eventually s_nexttime s_until s_until_with scalared sequence shortint
shortreal showcancelled signed small soft solve specify specparam static
string strong strong0 strong1 struct super supply0 supply1 sync_accept_on
sync_reject_on table tagged task this throughout time timeprecision timeunit
tran tranif0 tranif1 tri tri0 tri1 triand trior trireg type typedef union
unique unique0 unsigned until until_with untyped use uwire var vectored
virtual void wait wait_order wand weak weak0 weak1 while wildcard wire with
within wor xnor xor`

Это навигационный список, не грамматика допустимых сочетаний. Полный нормативный словарь и продукционные правила находятся в приложениях IEEE 1800. Новые роли слов в 2023 не обязательно добавляют новое зарезервированное слово.

Системные задачи и функции по назначению

Задача	Основные средства	Замечание
Печать	<code>\$display</code> , <code>\$write</code> , <code>\$strobe</code> , <code>\$monitor</code> , <code>\$monitoron</code> , <code>\$monitoroff</code>	Разный момент/условия печати
Ошибка и завершение	<code>\$info</code> , <code>\$warning</code> , <code>\$error</code> , <code>\$fatal</code> , <code>\$finish</code> , <code>\$stop</code> , <code>\$exit</code>	Код возврата зависит и от <code>flow</code>
Форматирование	<code>\$sformat</code> , <code>\$sformatf</code> , <code>\$swrite</code>	Строка, а не обязательно вывод в консоль

Задача	Основные средства	Замечание
Файлы	<code>\$fopen</code> , <code>\$fclose</code> , <code>\$fdisplay</code> , <code>\$fwrite</code> , <code>\$fgets</code> , <code>\$fscanf</code> , <code>\$sscanf</code>	Проверять статус
Доступ к файлу	<code>\$fgetc</code> , <code>\$ungetc</code> , <code>\$fread</code> , <code>\$feof</code> , <code>\$ferror</code> , <code>\$fflush</code> , <code>\$fseek</code> , <code>\$ftell</code> , <code>\$rewind</code>	EOF не заменяет проверку чтения
Память	<code>\$readmemh</code> , <code>\$readmemb</code> , <code>\$writememh</code> , <code>\$writememb</code>	Порядок адресов и расположение файла
Время	<code>\$time</code> , <code>\$stime</code> , <code>\$realtime</code> , <code>\$timeformat</code> , <code>\$printtimescale</code>	Масштаб и точность
Форма массива	<code>\$dimensions</code> , <code>\$unpacked_dimensions</code> , <code>\$left</code> , <code>\$right</code> , <code>\$low</code> , <code>\$high</code> , <code>\$increment</code> , <code>\$size</code>	Номер размерности
Тип/размер	<code>\$bits</code> , <code>\$typename</code> , <code>\$isunbounded</code>	Значение и тип — разные запросы
Биты	<code>\$countbits</code> , <code>\$countones</code> , <code>\$onehot</code> , <code>\$onehot0</code> , <code>\$isunknown</code>	X/Z имеют специальное значение
Преобразования	<code>\$signed</code> , <code>\$unsigned</code> , <code>\$cast</code> , <code>\$itor</code> , <code>\$rtoi</code>	Размер, знак и числовое значение
Представления real	<code>\$realtobits</code> , <code>\$bitstoreal</code> , <code>\$shortrealtobits</code> , <code>\$bitstoshortreal</code>	Битовая переинтерпретация
Математика	<code>\$clog2</code> , <code>\$ln</code> , <code>\$log10</code> , <code>\$exp</code> , <code>\$sqrt</code> , <code>\$pow</code> , <code>\$floor</code> , <code>\$ceil</code>	Не все вызовы — переносимый runtime RTL
Тригонометрия	<code>\$sin</code> , <code>\$cos</code> , <code>\$tan</code> , <code>\$asin</code> , <code>\$acos</code> , <code>\$atan</code> , <code>\$atan2</code> , гиперболические варианты	Обычно модели real
Случайность	<code>\$random</code> , <code>\$urandom</code> , <code>\$urandom_range</code> , <code>\$dist_*</code>	Signed/unsigned и потоки seed
Аргументы запуска	<code>\$test\$plusargs</code> , <code>\$value\$plusargs</code>	Воспроизводимость прогона
Трасса	<code>\$dumpfile</code> , <code>\$dumpvars</code> , <code>\$dumpon</code> , <code>\$dumpoff</code> , <code>\$dumpall</code> , <code>\$dumplimit</code> , <code>\$dumpflush</code>	Формат/поддержка инструмента
Выборка SVA	<code>\$sampled</code> , <code>\$past</code> , <code>\$rose</code> , <code>\$fell</code> , <code>\$stable</code> , <code>\$changed</code>	Работают с выбранными значениями
Контроль SVA	<code>\$asserton</code> , <code>\$assertoff</code> , <code>\$assertkill</code> , <code>\$assertcontrol</code> , семейства pass/fail/vacuous control	Проверки можно случайно отключить
Покрытие	<code>\$get_coverage</code> , <code>\$coverage_control</code> , <code>\$coverage_get</code> , <code>\$coverage_get_max</code> , <code>\$coverage_merge</code> , <code>\$coverage_save</code>	Поддержка и базы зависят от инструмента
Timing	<code>\$setup</code> , <code>\$hold</code> , <code>\$setuphold</code> , <code>\$recovery</code> , <code>\$removal</code> , <code>\$recrem</code> , <code>\$width</code> , <code>\$period</code>	Контекст specify
Аннотация	<code>\$sdf_annotate</code>	Имена иерархии и библиотека должны совпасть

В стандарте также есть средства очередей `$q_initialize/$q_add/$q_remove/ $q_full/$q_exam`, PLA-моделирования `$async$.../$sync$...` и глобальных `sampled-value` запросов. Они специализированы и не нужны для базового RTL; их точные сигнатуры нужно брать из соответствующего раздела LRM.

Часто используемые методы

string: len, getc, putc, tolower, toupper, compare, icompare, substr, atoi, atohex, atooc, atobin, atoreal, itoa, hextoa, octtoa, bintoa, realtoa. Индексы строк обращаются к байтам, не к произвольным Unicode-кодovým точкам; это важно для русского UTF-8.

Enum: first, last, next, prev, num, name. Динамический массив: size, delete. Очередь дополнительно: insert, push_front/back, pop_front/back. Ассоциативный массив: num, size, exists, first, last, next, prev, delete. Методы поиска, сортировки и свёрток разобраны в [главе 08](#).

Random: randomize, srandom, get_randstate, set_randstate, rand_mode, constraint_mode, callbacks pre_randomize/post_randomize. Синхронизация: mailbox, semaphore, process — классы/средства std, а не пользовательские аппаратные FIFO и арбитры.

Форматные спецификаторы

%b/%o/%d/%h — основания целого, %0d убирает лишнюю ширину поля, %s — строка, %c — символ, %f/%e/%g — real, %t — время, %p — агрегатное представление, %m — область/иерархическое имя. Не сравнивайте %p-строки разных симуляторов как нормативный формат протокола.

С. Упражнения и ответы

Сначала предскажите результат на бумаге, затем проверяйте его кодом. Если компилятор выдаёт предупреждение, объясните его причину до отключения. Для упражнений на аппаратную структуру дополнительно полезна схема и отчёт синтеза.

Ответы к главам 01–09

- 01.** Без фронта и сброса q не меняется; при $en=0$ не изменится и на фронте. Непрерывное `assign q=d` повторит изменения d после обработки событий модели.
- 02.** Значения: `unsigned 255`, `signed -1`, `1`. `'1` заполняет все 16 бит, а десятичная единица после преобразования даёт `16'h0001`.
- 03.** Максимум $255+255=510$ требует 9 бит. Именованное соединение сохраняет смысл после перестановки деклараций; позиционное может подключить другие порты.
- 04.** NBA-пара даёт $a=7$, $b=2$. Blocking-пара — $a=7$, $b=7$. Inactive с `#0` находится перед NBA, поэтому ожидание не является барьером NBA.
- 05.** `logic [12:0] r;`, `int unsigned count;`, `wire shared_line;`. `const`-переменная не обязательно является константой элаборации, в отличие от подходящего `parameter/localparam`.
- 06.** Первый сдвиг даёт `8'hC0` (-64), второй `8'h40` (64). Максимальная сумма трёх байтов 765 требует 10 бит: расширьте каждый вход до десяти бит перед сложением.
- 07.** Размер 45 бит. При порядке `valid, address, data: valid[44], address[43:32], data[31:0]`. Статический `cast enum` не проверяет допустимость метки.
- 08.** Packed-тип имеет 32 бита. У `mem` четыре элемента по 8 бит (всего также 32), но это другая организация. Для ожидаемых ответов удобна очередь транзакций.
- 09.** `typedef logic signed [15:0] s16_t;` и `s16_t'($signed(raw)).int'(1.8)` даёт 2, `$rtoi(1.8)` — 1.

Ответы к главам 10–18

- 10.** Неполное присваивание выражает хранение, обычно непреднамеренную защёлку. Исправление: `always_comb begin q='0; if (enable) q=d; end.`
- 11.** Обычный `case` выбирает первую подходящую ветвь; `unique` дополнительно диагностирует нарушение исключительности. `Priority` сохраняет смысл приоритета и не требует исключительности. Квалификатор не меняет сами маски.
- 12.** `Automatic` даёт отдельные аргументы/локальные объекты активным вызовам. `Output` копируется при возврате, запись через `ref` видна в исходном объекте во время выполнения. У `ref` важны совместимость типов и `lifetime`.
- 13.** Объявление должно быть доступно в месте использования; импорт в теле не перемещается назад в заголовок. `Bind` удобен для добавления одинаковой проверки ко всем экземплярам чужого или неизменяемого RTL.
- 14.** Адресные ширины: 1, 3, 4. Счётчики: 1, 3, 5. При глубине 5 трёхбитный указатель естественно оборачивается после 7, а нужно после 4.
- 15.** Источник выдаёт `request/address`, принимает `acknowledge`; приёмник наоборот. Экземпляр интерфейса создаёт свои объекты, одинаковые имена их не объединяют.

16. `ifdef` проверяет факт определения, а не числовое значение. Параметр можно независимо задать каждому экземпляру, сохранив типовую и элаборационную проверку.

17. `logic [8:0] full; assign full={1'b0,a}+{1'b0,b};` и `assign y=full[8] ? 8'hFF : full[7:0];`. Старший бит — `unsigned overflow`.

18. `Valid` проходит те же две ступени через `NBA` и сбрасывается в 0. Данные при `valid=0` не должны влиять на принимаемые транзакции или проверки значимости; тогда сбрасывать их ради функциональности не обязательно.

Ответы к главам 19–27

19. После операций `count`: 1, 2, 2, 1, 0. Пустота была до первой операции и после последней. Одновременные принятые чтение/запись сохраняют `count`.

20. Выбор контракта должен быть явным. Для удержания `done` оставьте автомат в `REPORT` до `ack`; выход `done` остаётся 1 всё это время. Новые `start` при занятости игнорируются или запрещаются контрактом/проверкой.

21. При переходе 01→10 независимые синхронизаторы могут дать 00 или 11. Для редкого слова удерживайте данные до подтверждения, а управление передавайте согласованным `handshake`; стабильность шины и `timing` также нужно обеспечить.

22. Только чтение: до фронта `full` запрещает запись. Для варианта замещения возможна формула `rst_n && (!full || (out_valid && out_ready))`, но она меняет комбинационные зависимости, RTL и эталон на границе.

23. `q != expected` может стать `X`; `if` не выбирает ветвь ошибки. `!==` даёт определённый ответ. Храните `seed`, версию симулятора, ревизию исходников и параметры/опции запуска.

24. Оставшийся таймер позже ошибочно завершит успешную операцию как `timeout`. `Disable fork` ограничивают отдельным родительским процессом, чтобы не убить другую фоновую работу вызывающей процедуры.

25. `Mailbox` передаёт `handle`, обе стороны видят объект. Создавайте новую транзакцию на передачу либо отправляйте независимую копию, включая нужные вложенные объекты.

26. Например `int unsigned length, bit [3:0] address` и ограничения `length inside {[1:4]}`; `int'(address)+length <= 16`; Порядок случайных вызовов, `solver` и код могут измениться даже при том же `seed`.

27. Для однократного `req` первая проверка требует `ack` в том же выбранном такте, вторая — в следующем. Без `req implication` проходит вакуумно; нужны `cover` на запрос и завершение сценария.

Ответы к главам 28–33

28. `Count` показывает объём, но не значения и их порядок. Нужен `scoreboard`. `Bin full` фиксирует заполненность; `cross/отдельное условие full && in_valid` фиксирует попытку записи, причём `handshake` при этом может отсутствовать.

29. Для известной C-функции естественен DPI. `Logic` имеет `X/Z`, для которых одного бита `bool` недостаточно; нужно стандартное DPI-представление.

30. Равносильные 0/1 дают `X`; 0/Z даёт 0, 1/Z — 1. Сеть разрешает драйверы, переменная не выполняет такого электрического разрешения записей.

- 31.** #2 — задержка модели, физический путь зависит от технологии и реализации. При ошибке SDF проверьте scope аннотации, имена иерархии/netlist, версию библиотеки и соответствие конкретной сборки файлу задержек.
- 32.** Instance точнее выбирает одну память. Нужны файловый список, библиотеки, include paths, defines, top/config, параметры, версия и опции инструмента.
- 33.** Противоречия нет: язык шире RTL-подмножества. Нужны подтверждённая функциональность и успешная реализация/анализ для целевой FPGA, включая ресурсы, timing и необходимые проверки пересечений доменов.

Практикумы с критериями приёмки

1. **ALU:** AND/OR/XOR и сложение с переносом. Полный перебор для ширины 4, проверка X-политики orcode, отсутствие latch, согласованная ширина результата.
2. **Конвейер:** задержка данных и valid на три такта. Сброс в середине потока, пустые такты, проверка точно заданной latency.
3. **FIFO:** прогоны глубин 1/3/8, wrap, full/empty, backpressure, reset непустого буфера, проверка порядка всех принятых элементов и автоматический timeout.
4. **Пакетный драйвер:** отдельные транзакции, mailbox, scoreboard, фиксируемый seed, корректная остановка процессов, отсутствие совместно изменяемых пакетов.
5. **Проверка протокола:** SVA удержания valid/data, ограниченного ответа, cover на активацию и завершение, аргументированные допущения окружения.

Успех практикума — выполненные критерии и объяснение результата, а не только синтаксически принятый код. Начальная база FIFO находится в [examples](#).

D. Соответствие исходному PLAN.md

Исходный план сохранён без изменений. Ниже перечислен каждый его нумерованный пункт и ближайшая глава новой методички. Мелкие пункты объединены в связанные объяснения; ссылка означает место рассмотрения темы, а не дословное повторение структуры или текста книги.

Сначала введены модули и симуляция, затем типы и выражения, после них процедуры и организация проекта. Практический RTL вынесен в отдельную часть, проверочные средства дополнены классами, randomization, SVA, coverage и DPI. Примитивы/конфигурации изучаются после рабочего RTL.

Таблица воспроизводится командой `python tools/map_plan.py`.

Глава 1. Предварительное знакомство с языком SystemVerilog

Пункт исходного плана	Новая глава
1.1. История языка SystemVerilog	01. Аппаратное мышление и устройство языка
1.2. Первоисточники языка SystemVerilog	33. Стандарты, изменения 2023 и рабочий процесс
1.3. Стандарты языка SystemVerilog	33. Стандарты, изменения 2023 и рабочий процесс
1.4. Базовые элементы языка SystemVerilog	02. Лексика, литералы и четыре состояния
1.4.1. Белые знаки	02. Лексика, литералы и четыре состояния
1.4.2. Комментарии	02. Лексика, литералы и четыре состояния
1.4.3. Операции	06. Операции, размер и знаковость
1.4.4. Идентификаторы	02. Лексика, литералы и четыре состояния
1.4.5. Ключевые слова	02. Лексика, литералы и четыре состояния
1.5. Логические значения	02. Лексика, литералы и четыре состояния
1.6. Числа	02. Лексика, литералы и четыре состояния
1.6.1. Представление целых чисел	02. Лексика, литералы и четыре состояния
1.6.2. Представление действительных чисел	02. Лексика, литералы и четыре состояния
1.7. Атрибуты	02. Лексика, литералы и четыре состояния
1.8. Параллелизм языка SystemVerilog	04. Время и планировщик симуляции
1.9. Синтез и моделирование проектов	01. Аппаратное мышление и устройство языка
1.9.1. Модель проекта для синтеза	01. Аппаратное мышление и устройство языка
1.9.2. Моделирование проекта	01. Аппаратное мышление и устройство языка
1.10. Конструктивные элементы языка SystemVerilog	01. Аппаратное мышление и устройство языка
1.10.1. Модули	03. Модули, порты и иерархия
1.10.2. Пакеты	13. Пакеты, области видимости и bind
1.10.3. Интерфейсы	15. Интерфейсы и modport
1.10.4. Примитивы	30. Примитивы, силы и пользовательские сети
1.10.5. Программы	23. Тестбенчи, clocking и ввод-вывод
1.10.6. Контролеры	27. Assertions, sequences и properties

Пункт исходного плана	Новая глава
1.10.7. Конфигурации	32. Библиотеки и конфигурации
1.11. Единицы компиляции	16. Препроцессор и единицы компиляции
1.12. Усовершенствования языка SystemVerilog для синтеза	33. Стандарты, изменения 2023 и рабочий процесс
1.13. Выводы	01. Аппаратное мышление и устройство языка

Глава 2. Модули

Пункт исходного плана	Новая глава
2.1. Определение модулей	03. Модули, порты и иерархия
2.2. Стили и уровни описания проектов	03. Модули, порты и иерархия
2.2.1. Структурный стиль описания проектов	03. Модули, порты и иерархия
2.2.2. Поведенческий стиль описания проектов	03. Модули, порты и иерархия
2.2.3. Уровни описания проекта	03. Модули, порты и иерархия
2.3. Элементы модулей	03. Модули, порты и иерархия
2.4. Объявления портов	03. Модули, порты и иерархия
2.5. Экземпляры модулей	03. Модули, порты и иерархия
2.6. Неявные соединения портов	03. Модули, порты и иерархия
2.6.1. Нотация .name неявного соединения портов	03. Модули, порты и иерархия
2.6.2. Нотация .* неявного соединения портов	03. Модули, порты и иерархия
2.7. Вложенные модули	03. Модули, порты и иерархия
2.8. Псевдонимы цепей	03. Модули, порты и иерархия
2.8.1. Оператор alias	03. Модули, порты и иерархия
2.8.2. Неявные объявления цепей в операторе alias	03. Модули, порты и иерархия
2.9. Использование псевдонимов цепей с нотациями неявных соединений .name и .*	03. Модули, порты и иерархия
2.10. Передача значений через порты модулей	03. Модули, порты и иерархия
2.11. Параметризованные модули	14. Параметры и generate
2.11.1. Объявление параметризованных модулей	14. Параметры и generate
2.11.2. Создание экземпляров параметризованных модулей	14. Параметры и generate
2.11.3. Параметризованные типы	14. Параметры и generate
2.12. Внешние модули (прототипы модулей)	32. Библиотеки и конфигурации
2.13. Выводы	03. Модули, порты и иерархия

Глава 3. Основные типы данных

Пункт исходного плана	Новая глава
3.1. Определения	05. Сети, переменные и встроенные типы
3.1.1. Типы данных и объекты данных	05. Сети, переменные и встроенные типы
3.1.2. Типы сингулярный, агрегированный, интегральный и простой битовый вектор	05. Сети, переменные и встроенные типы
3.1.3. Сети и переменные	05. Сети, переменные и встроенные типы
3.2. Сети	05. Сети, переменные и встроенные типы
3.2.1. Объявление сетей	05. Сети, переменные и встроенные типы
3.2.2. Задержки	31. specify, задержки и библиотечные модели
3.2.3. Типы сетей	30. Примитивы, силы и пользовательские сети
3.2.4. Логическая мощность сигналов цепей	30. Примитивы, силы и пользовательские сети
3.2.5. Значения сигналов сетей	30. Примитивы, силы и пользовательские сети
3.2.6. Особенности синтеза	A. Матрица синтезируемости
3.3. Переменные	05. Сети, переменные и встроенные типы
3.3.1. Явные и неявные объявления переменных	05. Сети, переменные и встроенные типы
3.3.2. Объявление переменных	05. Сети, переменные и встроенные типы
3.3.3. Инициализация переменных	05. Сети, переменные и встроенные типы
3.3.4. Переменные с 4 и с 2-мя состояниями	05. Сети, переменные и встроенные типы
3.3.5. Статические и автоматические переменные	05. Сети, переменные и встроенные типы
3.3.6. Инициализация статических и автоматических переменных	05. Сети, переменные и встроенные типы
3.3.7. Область действия и время жизни переменных	05. Сети, переменные и встроенные типы
3.4. Векторы	05. Сети, переменные и встроенные типы
3.4.1. Векторы и скаляры	05. Сети, переменные и встроенные типы
3.4.2. Выбор битов и битовых полей	05. Сети, переменные и встроенные типы
3.5. Целочисленные типы данных	05. Сети, переменные и встроенные типы
3.6. Типы данных действительных чисел	05. Сети, переменные и встроенные типы
3.7. Строковый тип данных	05. Сети, переменные и встроенные типы
3.8. Константы	05. Сети, переменные и встроенные типы
3.8.1. Константы языка SystemVerilog	05. Сети, переменные и встроенные типы
3.8.2. Константы parameter и localparam	05. Сети, переменные и встроенные типы
3.8.3. Константы specparam	31. specify, задержки и библиотечные модели
3.8.4. Константы const	05. Сети, переменные и встроенные типы
3.8.5. Знак \$ как значение параметра	05. Сети, переменные и встроенные типы
3.9. Выводы	05. Сети, переменные и встроенные типы

Глава 4. Пользовательские и перечисляемые типы

Пункт исходного плана	Новая глава
4.1. Пользовательские типы	07. typedef, enum, структуры и объединения
4.2. Перечисляемые типы	07. typedef, enum, структуры и объединения
4.2.1. Объявление переменных перечисляемого типа	07. typedef, enum, структуры и объединения
4.2.2. Базовый тип перечисляемых типов	07. typedef, enum, структуры и объединения
4.2.3. Автоматическая генерация меток перечисляемого типа	07. typedef, enum, структуры и объединения
4.2.4. Значения перечисляемого типа	07. typedef, enum, структуры и объединения
4.2.5. Введённые и анонимные перечисления	07. typedef, enum, структуры и объединения
4.2.6. Выполнение операций на переменных перечисляемого типа	07. typedef, enum, структуры и объединения
4.2.7. Приведение выражений к перечисляемому типу	07. typedef, enum, структуры и объединения
4.2.8. Специальные системные методы для перечисляемых типов	07. typedef, enum, структуры и объединения
4.2.9. Печать перечисляемых типов	07. typedef, enum, структуры и объединения
4.3. Выводы	07. typedef, enum, структуры и объединения

Глава 5. Совместимость и приведение типов

Пункт исходного плана	Новая глава
5.1. Совместимость типов	09. Совместимость и приведение типов
5.1.1. Совпадающие типы	09. Совместимость и приведение типов
5.1.2. Эквивалентные типы	09. Совместимость и приведение типов
5.1.3. Совместимость типов по назначению, по приведению и несовместимости типов	09. Совместимость и приведение типов
5.1.4. Совместимость сетевых типов	09. Совместимость и приведение типов
5.1.5. Системная функция \$typename	09. Совместимость и приведение типов
5.2. Приведение типов	09. Совместимость и приведение типов
5.2.1. Статическая операция приведения типов	09. Совместимость и приведение типов
5.2.2. Динамическая системная функция приведения типов \$cast	09. Совместимость и приведение типов
5.2.3. Приведение битовых потоков	09. Совместимость и приведение типов
5.2.4. Перечисляемые типы в числовых выражениях 1	09. Совместимость и приведение типов
5.2.5. Операция type 1	09. Совместимость и приведение типов
5.2.6. Системные функции преобразования типов 1	09. Совместимость и приведение типов
5.3. Выводы 1	09. Совместимость и приведение типов

Глава 6. Структуры и объединения

Пункт исходного плана	Новая глава
6.1. Структуры 1	07. typedef, enum, структуры и объединения
6.1.1. Объявление структур	07. typedef, enum, структуры и объединения
6.1.2. Шаблоны назначений	07. typedef, enum, структуры и объединения
6.1.3. Присваивание значений структурам	07. typedef, enum, структуры и объединения
6.1.4. Распакованные и упакованные структуры	07. typedef, enum, структуры и объединения
6.1.5. Передача структур через порты модулей	07. typedef, enum, структуры и объединения
6.1.6. Передача структур в качестве аргументов задачам и функциям	07. typedef, enum, структуры и объединения
6.2. Объединения	07. typedef, enum, структуры и объединения
6.2.1. Распакованные и упакованные объединения	07. typedef, enum, структуры и объединения
6.2.2. Теговые объединения	07. typedef, enum, структуры и объединения
6.3. Пример использования структур и объединений	07. typedef, enum, структуры и объединения
6.4. Выводы	07. typedef, enum, структуры и объединения

Глава 7. Массивы

Пункт исходного плана	Новая глава
7.1. Массивы в языке SystemVerilog	08. Массивы, очереди и методы
7.1.1. Упакованные массивы	08. Массивы, очереди и методы
7.1.2. Распакованные массивы	08. Массивы, очереди и методы
7.1.3. Инициализация массивов при объявлении	08. Массивы, очереди и методы
7.1.4. Присваивание значений массивам	08. Массивы, очереди и методы
7.1.5. Копирование массивов	08. Массивы, очереди и методы
7.1.6. Копирование массивов и структур с помощью приведения битовых потоков	08. Массивы, очереди и методы
7.1.7. Операции над массивами	08. Массивы, очереди и методы
7.1.8. Память	19. Память, ROM и очереди FIFO
7.1.9. Многомерные массивы	08. Массивы, очереди и методы
7.1.10. Обращение к элементам и фрагментам (частям) массивов	08. Массивы, очереди и методы
7.1.11. Передача массивов через порты и как аргументы задачам и функциям	08. Массивы, очереди и методы
7.1.12. Массивы, структуры и объединения	08. Массивы, очереди и методы
7.1.13. Реализация циклов над элементами массивов (оператор foreach)	08. Массивы, очереди и методы
7.1.14. Пример использования массивов	22. Сквозной проект: синхронный FIFO

Пункт исходного плана	Новая глава
7.2. Конкатенация распакованных массивов	08. Массивы, очереди и методы
7.3. Другие виды массивов	08. Массивы, очереди и методы
7.3.1. Динамические массивы	08. Массивы, очереди и методы
7.3.2. Ассоциативные массивы	08. Массивы, очереди и методы
7.3.3. Очереди	08. Массивы, очереди и методы
7.4. Системные функции и методы для работы с массивами	08. Массивы, очереди и методы
7.4.1. Системные функции	08. Массивы, очереди и методы
7.4.2. Системная функция \$bits	08. Массивы, очереди и методы
7.4.3. Методы обработки массивов	08. Массивы, очереди и методы
7.5. Выводы	08. Массивы, очереди и методы

Глава 8. Места объявлений элементов проекта

Пункт исходного плана	Новая глава
8.1. Пакеты	13. Пакеты, области видимости и bind
8.1.1. Определение пакетов	13. Пакеты, области видимости и bind
8.1.2. Ссылки на содержимое пакетов	13. Пакеты, области видимости и bind
8.1.3. Рекомендации по синтезу	13. Пакеты, области видимости и bind
8.1.4. Встроенный пакет std	13. Пакеты, области видимости и bind
8.2. Единицы компиляции	13. Пакеты, области видимости и bind
8.2.1. Внешние объявления единиц компиляции	13. Пакеты, области видимости и bind
8.2.2. Явный доступ к внешним объявлениям	13. Пакеты, области видимости и bind
8.2.3. Импорт пакетов в единицы компиляции	13. Пакеты, области видимости и bind
8.2.4. Условная компиляция пакетов	13. Пакеты, области видимости и bind
8.3. Объявления в процедурных блоках	13. Пакеты, области видимости и bind
8.4. Области видимости идентификаторов	13. Пакеты, области видимости и bind
8.5. Иерархические и точечные имена	13. Пакеты, области видимости и bind
8.6. Ссылки на имена вверх по иерархии	13. Пакеты, области видимости и bind
8.7. Привязка вспомогательного кода к контекстам или экземплярам модулей	13. Пакеты, области видимости и bind
8.8. Выводы	13. Пакеты, области видимости и bind

Глава 9. Операции и выражения

Пункт исходного плана	Новая глава
9.1. Выражения, операции и операнды	06. Операции, размер и знаковость

Пункт исходного плана	Новая глава
9.2. Константные выражения	06. Операции, размер и знаковость
9.3. Агрегатные выражения	06. Операции, размер и знаковость
9.4. Операции	06. Операции, размер и знаковость
9.4.1. Приоритет операций	06. Операции, размер и знаковость
9.4.2. Использование в выражениях целочисленных литералов	06. Операции, размер и знаковость
9.4.3. Операции с 2-мя и с 4-мя состояниями	06. Операции, размер и знаковость
9.4.4. Использование в выражениях операции назначения	06. Операции, размер и знаковость
9.5. Описание операций	06. Операции, размер и знаковость
9.5.1. Операции назначения	10. Присваивания и процессы RTL
9.5.2. Операции инкремента и декремента	06. Операции, размер и знаковость
9.5.3. Арифметические операции	06. Операции, размер и знаковость
9.5.4. Арифметические выражения со знаковыми и беззнаковыми типами	06. Операции, размер и знаковость
9.5.5. Операции отношения	06. Операции, размер и знаковость
9.5.6. Операции равенства	06. Операции, размер и знаковость
9.5.7. Операции равенства с подстановочными знаками	06. Операции, размер и знаковость
9.5.8. Логические операции	06. Операции, размер и знаковость
9.5.9. Побитовые операции	06. Операции, размер и знаковость
9.5.10. Операции редукции	06. Операции, размер и знаковость
9.5.11. Операции сдвига	06. Операции, размер и знаковость
9.5.12. Условная операция	06. Операции, размер и знаковость
9.5.13. Операция конкатенации	06. Операции, размер и знаковость
9.5.14. Операция репликации	06. Операции, размер и знаковость
9.5.15. Конкатенация строк	06. Операции, размер и знаковость
9.5.16. Операция членства в множестве	06. Операции, размер и знаковость
9.5.17. Поточковые операции	06. Операции, размер и знаковость
9.6. Размер битовых выражений	06. Операции, размер и знаковость
9.7. Знаковые выражения	06. Операции, размер и знаковость
9.8. Выражения строковых литералов	06. Операции, размер и знаковость
9.9. Конструкция let	06. Операции, размер и знаковость
9.10. Выводы	06. Операции, размер и знаковость

Глава 10. Операторы назначения

Пункт исходного плана	Новая глава
10.1. Назначения в языке SystemVerilog	10. Присваивания и процессы RTL
10.2. Непрерывные назначения	10. Присваивания и процессы RTL
10.3. Процедурные назначения	10. Присваивания и процессы RTL
10.3.1. Оператор блокирующего назначения (=)	10. Присваивания и процессы RTL
10.3.2. Оператор неблокирующего назначения (< =)	10. Присваивания и процессы RTL
10.3.3. Особенности синтеза процедурных операторов блокирующего и неблокирующего назначения	10. Присваивания и процессы RTL
10.4. Управление временем в процедурных операторах назначения	10. Присваивания и процессы RTL
10.4.1. Управление временем в операторах блокирующего назначения	10. Присваивания и процессы RTL
10.4.2. Внутренние задержки в операторах блокирующего назначения	10. Присваивания и процессы RTL
10.4.3. Управление временем в операторах неблокирующего назначения	10. Присваивания и процессы RTL
10.4.4. Внутренние задержки в операторах неблокирующего назначения	10. Присваивания и процессы RTL
10.5. Процедурные непрерывные назначения	10. Присваивания и процессы RTL
10.6. Выводы	10. Присваивания и процессы RTL

Глава 11. Процессы

Пункт исходного плана	Новая глава
11.1. Управление процедурным временем	04. Время и планировщик симуляции
11.1.1. Оператор управления задержками #	04. Время и планировщик симуляции
11.1.2. Оператор управления событиями @	04. Время и планировщик симуляции
11.1.3. Оператор ожидания wait	24. Параллельные процессы и синхронизация
11.1.4. Управление процедурным временем внутри назначения	10. Присваивания и процессы RTL
11.2. Процедурные блоки	24. Параллельные процессы и синхронизация
11.2.1. Последовательный блок begin-end	11. Условия, выбор и циклы
11.2.2. Параллельные блоки (fork-join, join any, join none)	24. Параллельные процессы и синхронизация
11.2.3. Время начала и время окончания процедурных блоков	24. Параллельные процессы и синхронизация
11.2.4. Имена процедурных блоков	11. Условия, выбор и циклы
11.2.6. Метки процедурных операторов	11. Условия, выбор и циклы
11.3. Структурные процедуры	10. Присваивания и процессы RTL
11.3.1. Процедура initial	10. Присваивания и процессы RTL
11.3.2. Процедуры always	10. Присваивания и процессы RTL

Пункт исходного плана	Новая глава
11.3.3. Процедура final	10. Присваивания и процессы RTL
11.4. Управление процессами	24. Параллельные процессы и синхронизация
11.4.1. Оператор wait fork	24. Параллельные процессы и синхронизация
11.4.2. Оператор wait order	24. Параллельные процессы и синхронизация
11.4.3. Оператор disable	24. Параллельные процессы и синхронизация
11.4.4. Оператор disable fork	24. Параллельные процессы и синхронизация
11.4.5. Тонкое управление процессами	24. Параллельные процессы и синхронизация
11.5. Выводы	24. Параллельные процессы и синхронизация

Глава 12. Процедурные операторы

Пункт исходного плана	Новая глава
12.1. Условный оператор if-else	11. Условия, выбор и циклы
12.1.1. Конструкция if-else-if	11. Условия, выбор и циклы
12.1.2. Квалификаторы unique, unique0 и priority	11. Условия, выбор и циклы
12.1.3. Особенности синтеза квалификаторов unique, unique0, priority в конструкции if-else-if	11. Условия, выбор и циклы
12.2. Оператор case	11. Условия, выбор и циклы
12.2.1. Операторы casez и casex	11. Условия, выбор и циклы
12.2.2. Константные выражения в операторах case	11. Условия, выбор и циклы
12.2.3. Квалификаторы unique, unique0 и priority в операторах case	11. Условия, выбор и циклы
12.2.4. Особенности синтеза квалификаторов unique, unique0, priority в операторе case	11. Условия, выбор и циклы
12.2.5. Атрибуты синтеза full case и parallel case	11. Условия, выбор и циклы
12.2.6. Атрибут full case	11. Условия, выбор и циклы
12.2.7. Атрибут parallel case	11. Условия, выбор и циклы
12.2.8. Сравнение квалификаторов unique и priority с атрибутами full case и parallel case и конструкциями else и default	11. Условия, выбор и циклы
12.2.9. Квалификатор inside оператора case	11. Условия, выбор и циклы
12.3. Операторы циклов	11. Условия, выбор и циклы
12.3.1. Цикл for	11. Условия, выбор и циклы
12.3.2. Цикл repeat	11. Условия, выбор и циклы
12.3.3. Цикл foreach	11. Условия, выбор и циклы
12.3.4. Цикл while	11. Условия, выбор и циклы
12.3.5. Цикл do-while	11. Условия, выбор и циклы
12.3.6. Цикл forever	11. Условия, выбор и циклы

Пункт исходного плана	Новая глава
12.5. Операторы переходов	11. Условия, выбор и циклы
12.5.1. Оператор continue	11. Условия, выбор и циклы
12.5.2. Оператор break	11. Условия, выбор и циклы
12.5.3. Оператор return ⁴	11. Условия, выбор и циклы
12.6. Выводы	11. Условия, выбор и циклы

Глава 13. Задачи и функции

Пункт исходного плана	Новая глава
13.1. Общая информация	12. Функции, задачи и время жизни
13.2. Автоматические и статические задачи и функции	12. Функции, задачи и время жизни
13.3. Задачи	12. Функции, задачи и время жизни
13.3.1. Объявление задач	12. Функции, задачи и время жизни
13.3.2. Вызов задач	12. Функции, задачи и время жизни
13.4. Функции	12. Функции, задачи и время жизни
13.4.1. Объявление функций	12. Функции, задачи и время жизни
13.4.2. Возвращаемые значения	12. Функции, задачи и время жизни
13.4.3. Аргументы функций типа output и inout	12. Функции, задачи и время жизни
13.4.4. Функции типа void	12. Функции, задачи и время жизни
13.4.5. Рекурсивные функции	12. Функции, задачи и время жизни
13.5. Константные функции	12. Функции, задачи и время жизни
13.6. Вызовы подпрограмм и передача аргументов	12. Функции, задачи и время жизни
13.6.1. Передача аргументов по значению	12. Функции, задачи и время жизни
13.6.2. Передача аргументов по ссылке	12. Функции, задачи и время жизни
13.6.3. Значения аргументов по умолчанию	12. Функции, задачи и время жизни
13.6.4. Передача аргументов по имени	12. Функции, задачи и время жизни
13.6.5. Передача в качестве аргументов массивов, структур и объединений	12. Функции, задачи и время жизни
13.7. Пустые задачи и функции	12. Функции, задачи и время жизни
13.8. Параметризованные задачи и функции	12. Функции, задачи и время жизни
13.9. Сравнение задач и функций	12. Функции, задачи и время жизни
13.10. Системные задачи и функции	В. Справочник ключевых слов и системных средств
13.10.1. Функции для работы с битовыми выражениями	В. Справочник ключевых слов и системных средств
13.11. Выводы	12. Функции, задачи и время жизни

Глава 14. Интерфейсы

Пункт исходного плана	Новая глава
14.1. Общая информация	15. Интерфейсы и modport
14.2. Объявление интерфейсов	15. Интерфейсы и modport
14.2.1. Пример цифровой системы без использования интерфейсов	15. Интерфейсы и modport
14.2.2. Группирование сигналов с помощью интерфейсов	15. Интерфейсы и modport
14.2.3. Порты интерфейсов	15. Интерфейсы и modport
14.2.4. Использование нотаций .name и *	15. Интерфейсы и modport
14.2.5. Глобальные и локальные интерфейсы	15. Интерфейсы и modport
14.3. Использование интерфейсов в качестве портов модулей	15. Интерфейсы и modport
14.3.1. Явно объявленные интерфейсные порты	15. Интерфейсы и modport
14.3.2. Универсальные интерфейсные порты	15. Интерфейсы и modport
14.4. Создание экземпляров интерфейсов и подключение интерфейсов к портам модулей	15. Интерфейсы и modport
14.4.1. Создание экземпляров интерфейсов	15. Интерфейсы и modport
14.4.2. Подключение интерфейсов к портам модулей	15. Интерфейсы и modport
14.4.3. Подключение интерфейсов к другим интерфейсам	15. Интерфейсы и modport
14.5. Ссылка на сигналы в пределах интерфейса	15. Интерфейсы и modport
14.6. Модпорты интерфейсов	15. Интерфейсы и modport
14.6.1. Определения модпортов	15. Интерфейсы и modport
14.6.2. Два способа подключения интерфейсов с модпортами к экземплярам модулей	15. Интерфейсы и modport
14.6.3. Ограничение доступа к сигналам интерфейса с помощью модпортов	15. Интерфейсы и modport
14.6.4. Сравнение различных способов описания интерфейсов	15. Интерфейсы и modport
14.7. Использование в интерфейсах задач и функций	15. Интерфейсы и modport
14.7.1. Интерфейсные методы	15. Интерфейсы и modport
14.7.2. Импорт интерфейсных методов в модпорты	15. Интерфейсы и modport
14.7.3. Экспорт интерфейсных методов	15. Интерфейсы и modport
14.8. Использование в интерфейсах процедурных блоков	15. Интерфейсы и modport
14.9. Параметризованные интерфейсы	15. Интерфейсы и modport
14.10. Несинтезируемые конструкции интерфейсов	23. Тестбенчи, clocking и ввод-вывод
14.11. Выводы	15. Интерфейсы и modport

Глава 15. Генерация кода

Пункт исходного плана	Новая глава
15.1. Синтаксис конструкций генерации	14. Параметры и generate
15.2. Конструкции генерации циклов	14. Параметры и generate
15.3. Конструкции условной генерации	14. Параметры и generate
15.4. Внешние имена неименованных блоков генерации	14. Параметры и generate
15.5. Выводы	14. Параметры и generate

Глава 16. Директивы компиляции

Пункт исходного плана	Новая глава
16.1. Возврат к умалчиваемым значениям директив компиляции	16. Препроцессор и единицы компиляции
16.2. Определение значения единицы времени	16. Препроцессор и единицы компиляции
16.3. Макроопределения	16. Препроцессор и единицы компиляции
16.3.1. Подстановка макросов в текстовую строку	16. Препроцессор и единицы компиляции
16.3.2. Создание из макросов имён идентификаторов	16. Препроцессор и единицы компиляции
16.4. Директивы условной компиляции	16. Препроцессор и единицы компиляции
16.5. Определение умалчиваемого типа цепей	16. Препроцессор и единицы компиляции
16.6. Определение логических значений для неподсоединённых входов	16. Препроцессор и единицы компиляции
16.7. Обозначение модулей как модули ячеек	16. Препроцессор и единицы компиляции
16.8. Прагмы0	16. Препроцессор и единицы компиляции
16.9. Включение файлов	16. Препроцессор и единицы компиляции
16.10. Определение номеров строк исходного кода	16. Препроцессор и единицы компиляции
16.11. Директивы FILE и LINE	16. Препроцессор и единицы компиляции
16.12. Директивы begin keywords и end keywords	16. Препроцессор и единицы компиляции
16.13. Выводы	16. Препроцессор и единицы компиляции

Глава 17. Примитивы

Пункт исходного плана	Новая глава
17.1. Где можно найти готовое решение	33. Стандарты, изменения 2023 и рабочий процесс
17.2. Примитивы языка SystemVerilog	30. Примитивы, силы и пользовательские сети
17.2.1. Типы примитивов	30. Примитивы, силы и пользовательские сети
17.2.2. Сила драйвера	30. Примитивы, силы и пользовательские сети
17.2.3. Задержки	31. specify, задержки и библиотечные модели
17.2.4. Массивы примитивов	30. Примитивы, силы и пользовательские сети

Пункт исходного плана	Новая глава
17.2.5. Список терминалов	30. Примитивы, силы и пользовательские сети
17.2.6. Логические вентили and, nand, or, nor, xor и xnor	30. Примитивы, силы и пользовательские сети
17.2.7. Буферы buf и not	30. Примитивы, силы и пользовательские сети
17.2.8. Буферы bufif1, bufif0, notif1 и notif0	30. Примитивы, силы и пользовательские сети
17.2.9. Переключатели MOS	30. Примитивы, силы и пользовательские сети
17.2.10. Двухнаправленные переключатели	30. Примитивы, силы и пользовательские сети
17.2.11. Источники pullup и pulldown	30. Примитивы, силы и пользовательские сети
17.3. Пользовательские примитивы	30. Примитивы, силы и пользовательские сети
17.3.1. Объявление пользовательских примитивов	30. Примитивы, силы и пользовательские сети
17.3.2. Комбинационные пользовательские примитивы	30. Примитивы, силы и пользовательские сети
17.3.3. Последовательностные пользовательские примитивы, чувствительные к уровню управляющего сигнала	30. Примитивы, силы и пользовательские сети
17.3.4. Последовательностные пользовательские примитивы, чувствительные к фронту управляющего сигнала	30. Примитивы, силы и пользовательские сети
17.3.5. Инициализация последовательностных пользовательских примитивов	30. Примитивы, силы и пользовательские сети
17.3.6. Пользовательские примитивы, чувствительные как к уровню, так и к фронту управляющего сигнала	30. Примитивы, силы и пользовательские сети
17.3.7. Создание экземпляров пользовательских примитивов	30. Примитивы, силы и пользовательские сети
17.4. Выводы	30. Примитивы, силы и пользовательские сети

Глава 18. Конфигурация проекта

Пункт исходного плана	Новая глава
18.1. Конфигурации	32. Библиотеки и конфигурации
18.2. Конфигурационные блоки	32. Библиотеки и конфигурации
18.3. Файл карты библиотеки	32. Библиотеки и конфигурации
18.4. Иерархические конфигурации	32. Библиотеки и конфигурации
18.5. Указание в конфигурации значений параметров модулей	32. Библиотеки и конфигурации
18.6. Примеры конфигурации проекта	32. Библиотеки и конфигурации
18.6.1. Исходное описание проекта	32. Библиотеки и конфигурации
18.6.2. Использование конфигурации, заданной в файле карты библиотеки	32. Библиотеки и конфигурации
18.6.3. Использование оператора default	32. Библиотеки и конфигурации
18.6.4. Использование оператора cell	32. Библиотеки и конфигурации
18.6.5. Использование оператора instance	32. Библиотеки и конфигурации
18.6.6. Использование иерархической конфигурации	32. Библиотеки и конфигурации
18.7. Выводы	32. Библиотеки и конфигурации

Дополнения и нумерованные материалы

Предисловие и маршрут чтения — глава 01. Практическое завершение курса — главы 22 и 33. Ключевые слова и системные средства — приложение В, ответы — С, литература — Е, сокращения и предметный указатель — F.

По сравнению с исходным планом добавлены самостоятельные главы о RAM/FIFO, автоматах, CDC/RDC, тестбенчах и clocking, классах, constrained random, assertions, coverage, DPI/VPI, specify/SDF и изменениях IEEE 1800-2023.

Всего сопоставлено нумерованных пунктов: **362**.

Е. Источники и порядок проверки фактов

Дата проверки ссылок и изменяемых сведений: **2026-09-23**. Изложение и учебные примеры написаны самостоятельно. Исходный PLAN.md используется как перечень тем, не как источник текста книги. Платный учебник не нужен для сборки и чтения.

Нормативные и первичные источники

Источник	Для чего нужен
IEEE 1800-2023: карточка и GET	Статус, дата публикации, доступ к нормативному LRM
IEEE Xplore: документ 10458102	Официальная публикация стандарта
Accellera: бесплатный доступ, 04.03.2024	Подтверждение участия в IEEE GET
Accellera UVM	Библиотека и стандарт методологии отдельно от языка
Slang manual	Команды и назначение компилятора
Slang v6.0	Первичная карта внедрения новых возможностей 2023
Slang v7.0	Завершение поддержки обновления в этой реализации
Verilator: языковые возможности	Отличия реализации и ограничения моделирования
Icarus Verilog	Исходники и сведения об открытом симуляторе
AMD UG901	Пример vendor-документации синтезируемого подмножества
ReportLab: Platypus	Реализация постраничной сборки PDF
ReportLab: paragraphs	Кириллический текст, форматирование и ссылки

Как разрешать сомнения

Для точной семантики сначала найдите конструкцию в IEEE LRM, затем проверьте errata/разъяснения и документацию конкретного инструмента. После этого создайте минимальный пример с понятным ожидаемым результатом. Принятие примера компилятором не делает расширение производителя частью стандарта.

Для синтеза дополнительно задайте целевую технологию, версию, атрибуты и ограничения. Изучите отчёт выведенных примитивов, число регистров, RAM/DSP и предупреждения. Для временной семантики запишите области выбора/обновления и проверьте трассу событий, а не только итоговую картинку.

Нормативный стандарт большой: полная BNF, правила редких комбинаций типов, все таблицы переключательных примитивов и API VPI требуют обращения к оригиналу. Методичка даёт учебную последовательность, рабочие модели и указатели на эти области, не пытаясь заменить все таблицы LRM сокращёнными пересказами.

Обновление методички

Новая дата документации EDA не обязательно означает новую редакцию языка. При обновлении проверяйте отдельно карточку IEEE, возможности compiler frontend, runtime симулятора и синтезатора. Записывайте конкретные версии в отчёт проверки, обновляйте примеры и пересобирайте PDF из исходных глав. Непроверенное изменение нельзя обозначать как подтверждённую совместимость.

Г. Словарь и предметный указатель

Словарь

Термин	Значение
HDL	Язык описания аппаратуры
RTL	Представление передачи и обработки данных между регистрами
DUT	Проверяемое устройство
Testbench	Окружение стимулов, наблюдения и проверок
Elaboration / элаборация	Построение иерархии с параметрами до моделирования
Driver	Источник значения сети; в тестбенче также компонент подачи стимулов
Net / сеть	Объект, получающий значение от драйверов по правилам разрешения
Variable / переменная	Объект, сохраняющий последнее присвоенное значение
Packed	Единое битовое представление агрегата
Unpacked	Агрегат самостоятельных элементов
Handle	Ссылка на программный объект или ресурс
Scope	Область видимости имени
Lifetime	Время существования объекта
NBA	Очередь неблокирующих обновлений
Latch	Защёлка, прозрачная при активном уровне управления
Flip-flop	Триггер, обновляемый по событию фронта
CDC / RDC	Пересечение тактовых доменов / доменов сброса
STA	Статический временной анализ
Latency	Задержка от принятия входа до соответствующего выхода
Throughput	Установившаяся пропускная способность
Backpressure	Приостановка источника неготовностью потребителя
Scoreboard	Проверка наблюдаемых транзакций по эталонной модели
Assertion	Исполняемое утверждение о поведении
Vacuity	Успех требования без активации существенной предпосылки
Coverage	Измерение наблюденных запланированных ситуаций
Seed	Начальное состояние генератора случайности
LRM	Нормативное описание языка
DPI / VPI	Прямой интерфейс подпрограмм / программный доступ к объектам модели

Указатель по задачам

Искомая тема	Где читать
Четыре состояния, числа, строки, идентификаторы	02
Модуль, порт, .name, .*, alias	03
Время, Active/NBA, гонки	04
wire, logic, bit, signed, const	05
Переполнение, операторы, inside, let, streaming	06
Enum, struct, union, assignment pattern	07
Массив, queue, associative, foreach, sum	08
Cast, \$cast, \$signed, bit-stream	09
assign, =, <=, always_comb/ff/latch	10
if, case, unique, priority, циклы	11
function/task, ref, automatic/static	12
Package, import/export, scope, bind	13
Параметры, generate, genvar, \$clog2	14
Interface, modport, virtual interface	15
Макросы, include, directives, compilation unit	16
Мультиплексор, арифметика, fixed point	17
Регистр, reset, enable, pipeline	18
RAM/ROM, read-during-write, FIFO	19, 22
FSM, Moore/Mealy, handshake	20
Синхронизатор, CDC, timing constraints	21
Тестбенч, clocking, файлы, plusargs	23
Fork, event, mailbox, semaphore, process	24
Class, inheritance, constructor, copy	25
Random, constraint, dist, randcase, randsequence	26
SVA, sequence, property, sampled values	27
Covergroup, bins, cross, functional coverage	28
DPI-C, VPI, UVM	29
UDP, силы драйверов, MOS, nettype	30
Specify, SDF, timing checks	31
Config, library, extern	32
IEEE 1800-2023, инструменты	33

Указатель полных примеров

Компилируемые примеры и команды собраны в [examples/README.md](#). Сквозной FIFO соединяет темы глав 03–22. `core.sv` проверяет типы и выражения, `verification.sv` демонстрирует проверочную модель, `primitives.sv` — UDP/specify, `sv2023.sv` — отдельные дополнения новой редакции. Фактический статус каждого набора проверок — [VALIDATION.md](#).